



UNIVERSIDAD DE TALCA
FACULTAD DE INGENIERÍA
ESCUELA DE INGENIERÍA CIVIL EN COMPUTACIÓN

**Sistema IoT de asistencia con autenticación
biométrica e integración empresarial**

AGUSTIN EDUARDO MEZA MELLA

Profesor Guía: RICARDO PÉREZ GUZMÁN

Memoria para optar al título de
Ingeniero Civil en Computación Curicó, Chile— Noviembre, 2026



UNIVERSIDAD DE TALCA
FACULTAD DE INGENIERÍA
ESCUELA DE INGENIERÍA CIVIL EN COMPUTACIÓN

**Sistema IoT de asistencia con autenticación
biométrica e integración empresarial**

AGUSTIN EDUARDO MEZA MELLA

Profesor Guía: RICARDO PÉREZ GUZMÁN

El presente documento fue calificado con nota: _____

Memoria para optar al título de
Ingeniero Civil en Computación Curicó, Chile— Noviembre, 2026

TABLA DE CONTENIDOS

Tabla de Contenidos	1
Índice de Figuras	4
Índice de Tablas	5
Resumen	6
1 Introducción	8
1.1 Contexto	8
1.2 Descripción del problema	9
1.3 Propuesta de solución	10
1.4 Objetivos	10
1.4.1 Objetivo general	11
1.4.2 Objetivos específicos	11
1.5 Alcances	12
1.6 Resumen capítulo	13
2 Marco teórico	14
2.1 Conceptos básicos	14
2.1.1 Conceptos globales	14
2.1.2 Conceptos de software	15
2.2 Tecnologías utilizadas	17
2.2.1 Microcontrolador ESP32-CAM	17
2.2.2 Lector de huella digital AS608	17
2.2.3 Protocolos de comunicación HTTP	17
2.2.4 Protocolo de comunicación MQTT	18
2.2.5 Contenedores Docker	18
2.2.6 Sistemas de bases de datos	18
2.2.7 Sensor PIR (HC-SR501)	19
2.2.8 Framework DeepFace	19
2.2.9 Broker Mosquitto en contenedor Docker	19
2.3 Estado del arte	20
2.3.1 Soluciones basadas en hardware	20
2.3.2 Soluciones basadas en software	21
2.4 Metodologías	22
2.4.1 Metodología de desarrollo	22
2.4.2 Metodología de evaluación	23

2.5	Resumen del capítulo	24
3	Metodologías	25
3.1	Metodología de desarrollo	25
3.1.1	Protipado Evolutivo Incremental	25
3.2	Planificación	27
3.2.1	Iteración 1: Integración de hardware y servidor embebido	27
3.2.2	Iteración 2: Almacenamiento local y modo offline	28
3.2.3	Iteración 3: Backend, base de datos y comunicación	29
3.2.4	Iteración 4: Reconocimiento facial, anti-spoofing y cifrado biométrico	31
3.2.5	Iteración 5: Módulo antifraude y validación previa a la captura	32
3.2.6	Iteración 6: Autenticación, multi-tenant y enrolamiento de dispositivos	34
3.2.7	Iteración 7: Panel web para la gestión del dispositivo e integración con ERP	35
3.2.8	Iteración 8: Sincronización diferida, logs y cierre del proyecto . .	37
3.3	Metodología de evaluación	39
3.3.1	Pruebas de integración	39
3.3.2	Pruebas de usabilidad	40
3.3.3	Pruebas de confiabilidad y disponibilidad	42
3.4	Resultados Esperados	42
3.5	Resumen del capítulo	43
4	Desarrollo	44
4.1	Diseño de software	44
4.1.1	Arquitectura física	44
4.1.2	Arquitectura lógica	45
4.2	Implementación	45
4.3	Iteración 1: Integración de hardware y servidor embebido	46
4.3.1	Análisis	46
4.3.2	Diseño	47
4.3.3	Implementación	51
4.3.4	Pruebas	60
4.4	Iteración 2: Almacenamiento local y modo offline	61
4.4.1	Análisis	61
4.4.2	Diseño	62
4.4.3	Implementación	62
4.4.4	Pruebas	65
4.5	Iteración 3: Backend, base de datos y comunicación	66
4.5.1	Análisis	66

4.5.2	Diseño	67
4.5.3	Implementación	69
4.5.4	Pruebas	72
4.6	Iteración 4: Reconocimiento facial, anti-spoofing y cifrado biométrico .	73
4.6.1	Análisis	73
4.6.2	Diseño	74
4.6.3	Implementación	76
4.6.4	Pruebas	77
4.7	Iteración 5: Autenticación, multi-tenant y enrolamiento de dispositivos	78
4.7.1	Análisis	78
4.7.2	Diseño	79
4.7.3	Implementación	81
4.7.4	Pruebas	82
4.8	Iteración 6: Módulo antifraude y validación previa a la captura	83
4.8.1	Análisis	84
4.8.2	Diseño	85
4.8.3	Implementación	86
4.8.4	Pruebas	87
4.9	Iteración 7: Panel web backend e integración con ERP	88
4.9.1	Análisis	89
4.9.2	Diseño	89
4.9.3	Implementación	91
4.9.4	Pruebas	92
4.10	Iteración 8: Sincronización diferida, logs y cierre del proyecto	93
4.10.1	Análisis	94
4.10.2	Diseño	94
4.10.3	Implementación	96
4.10.4	Pruebas	98
4.11	Resumen del capítulo	99

ÍNDICE DE FIGURAS

2.1	Metodología prototipado evolutivo con enfoque incremental	23
4.1	Arquitectura física	44
4.2	Arquitectura lógica	45
4.3	Mockup menú principal en el ESP32-CAM	47
4.4	Mockup panel de configuración Wi-Fi	48
4.5	Mockup registro usuario en el ESP32-CAM	48
4.6	Mockup usuarios almacenados en el ESP32-CAM	49
4.7	Mockup visualización de turnos en el ESP32-CAM	49
4.8	Mockup creación de turnos en el ESP32-CAM	50
4.9	Mockup asignación de turnos en el ESP32-CAM	50
4.10	Flujo operacional para el registro de asistencia.	51
4.11	Conexiones entre el ESP32-CAM y el lector de huellas AS608.	52
4.12	Fragmento de código para inicializar el lector AS608 utilizando UART.	53
4.13	Asignación de rutas para el servidor web embebido.	55
4.14	Menú principal del sistema web embebido.	56
4.15	Interfaz de configuración Wi-Fi.	56
4.16	Vista para el registro de personas en el dispositivo.	57
4.17	Vista de asistencias locales almacenadas en el dispositivo.	58
4.18	Vista de personas registradas en el dispositivo.	58
4.19	Vista de gestión de turnos en el dispositivo.	59
4.20	Vista de gestión de asignaciones en el dispositivo.	60
4.21	Funciones para lectura y escritura de archivos JSON en LittleFS.	63
4.22	Diagrama de la base de datos relacional.	69

ÍNDICE DE TABLAS

RESUMEN

El presente proyecto aborda el desarrollo de un sistema IoT de control de asistencia biométrica, diseñado para responder a las limitaciones actuales de las soluciones comerciales y cumplir con los requisitos técnicos y legales establecidos por la Dirección del Trabajo de Chile, en particular la Resolución Exenta N.º38 (2024), que regula los sistemas electrónicos de registro de asistencia. El trabajo propone una alternativa modular, de bajo costo, código abierto y adaptable, orientada principalmente a pequeñas y medianas empresas que requieren un sistema flexible, con capacidad de funcionamiento autónomo y con posibilidades reales de integración con plataformas ERP.

La solución desarrollada utiliza como dispositivo central un ESP32-CAM, microcontrolador que incorpora cámara, conectividad Wi-Fi y capacidades de procesamiento suficientes para realizar captura de imágenes y ejecutar tareas de apoyo asociadas a la autenticación. A este módulo se integra un lector de huellas AS608, que permite una verificación biométrica complementaria frente a situaciones donde la captura facial no sea óptima. El sistema se estructura de manera híbrida, permitiendo registrar asistencia tanto mediante reconocimiento facial como mediante verificación dactilar, de acuerdo con las condiciones de operación.

Uno de los principales aportes del proyecto es la capacidad del dispositivo para operar en modo offline, utilizando SPIFFS como sistema de almacenamiento local para gestionar usuarios, turnos, asignaciones y asistencias en formato JSON. Cuando la conectividad se restablece, el sistema ejecuta un proceso de sincronización diferida que garantiza la integridad de los datos y la continuidad operativa del dispositivo, incluso en entornos con conectividad inestable o intermitente, situación común en empresas pequeñas o en faenas remotas. Asimismo, la arquitectura lógica y física diseñada permite incorporar módulos de seguridad adicionales, como un sensor PIR combinado con anti-spoofing por software, destinado a detectar intentos de suplantación mediante fotografías o pantallas.

El desarrollo del proyecto se llevó a cabo utilizando una metodología de prototipado evolutivo incremental, seleccionada por su capacidad para integrar hardware, firmware, backend y módulos biométricos de manera progresiva. A lo largo de ocho iteraciones se construyeron y validaron componentes funcionales independientes que luego fueron integrados: capturas biométricas, almacenamiento local, comunicación HTTP/MQTT, panel web embebido, mecanismos antifraude, backend con base de datos PostgreSQL y módulo de comunicación estándar para ERP. Cada iteración incluyó análisis, diseño, implementación y pruebas, permitiendo detectar fallas tempranas y optimizar el prototipo de manera continua.

La etapa de evaluación incorporó pruebas de integración, usabilidad, confiabilidad y

disponibilidad, fundamentales para determinar el desempeño del sistema en escenarios reales. Las pruebas de integración verificaron la correcta comunicación entre los distintos módulos del prototipo, incluyendo la interoperabilidad entre el ESP32-CAM, el lector AS608, el almacenamiento SPIFFS, el backend y la base de datos. Las pruebas de usabilidad, aplicadas tanto a usuarios técnicos como no técnicos mediante el método System Usability Scale (SUS), permitieron evaluar la claridad de las interfaces y la facilidad de uso del sistema. Complementariamente, se desarrollaron pruebas de confiabilidad y disponibilidad destinadas a medir la robustez del sistema frente a desconexiones, reconexiones y funcionamiento prolongado sin supervisión.

El sistema resultante constituye un prototipo funcional que permite registrar asistencia, almacenar datos localmente, sincronizar registros con un backend cuando existe conectividad, operar de manera continua bajo distintos escenarios y comunicar información hacia sistemas ERP mediante una API estándar. A través del análisis de los resultados obtenidos, se concluye que la solución propuesta es técnicamente viable, cumple con los requisitos de funcionamiento planteados en los objetivos específicos y representa una alternativa accesible y adaptable frente a las restricciones de los dispositivos biométricos comerciales actuales.

1. Introducción

En el siguiente capítulo se presenta el contexto, la problemática actual y la propuesta de solución que fundamentan el desarrollo de este sistema IoT de control de asistencia biométrico.

1.1 Contexto

El control de asistencia laboral es un requisito obligatorio para todas las empresas en Chile, conforme a lo dispuesto por la Dirección del Trabajo. Este registro puede realizarse de manera manual o mediante sistemas digitales que cumplan con la Resolución Exenta N°38 (2024) [1], la cual establece los requisitos técnicos y legales que deben cumplir los sistemas electrónicos para ser considerados válidos ante la autoridad.

El cumplimiento de estas normas no solo busca garantizar la trazabilidad y transparencia de las jornadas laborales, sino que también impacta directamente en el cálculo de remuneraciones y en la gestión administrativa de la empresa[2], si no se lleva un registro que cumpla las características presentes en la norma, tales como la inclusión de métodos que garanticen el registro de asistencia al trabajador. Por ello, muchas organizaciones optan por integrar sus sistemas de asistencia con plataformas ERP (EntERPrise Resource Planning), las cuales centralizan distintos procesos como recursos humanos, inventario, finanzas y contabilidad dentro de un mismo entorno[3].

Esta integración permite que los datos de asistencia alimenten automáticamente los módulos de remuneraciones o de gestión de personal, evitando errores y reduciendo los tiempos administrativos[4]. En este contexto, los sistemas de asistencia pueden ser parte del propio ERP o soluciones externas especializadas, las cuales se comunican mediante APIs o servicios web.

Las máquinas de control de asistencia digital, por su parte, permiten registrar las marcaciones de entrada y salida de los trabajadores de forma autónoma[5]. Sin embargo, muchas de las soluciones disponibles en el mercado presentan limitaciones: operan únicamente en red local, exportan datos en formatos cerrados (como CSV o Excel) y

no siempre permiten integrarse de manera flexible con otros sistemas, como los ERP corporativos.

Este proyecto propone el desarrollo de un sistema IoT de control de asistencia basado en ESP32-CAM, capaz de realizar reconocimiento facial y enviar los registros al backend, permitiendo su posterior integración con un ERP o funcionamiento independiente en modo local.

1.2 Descripción del problema

Aunque la normativa chilena exige que los sistemas de control de asistencia garanticen trazabilidad, integridad y disponibilidad de los registros de acuerdo con la Resolución Exenta N.º 38 (2024), una parte importante de los dispositivos utilizados actualmente especialmente los relojes de control biométricos de bajo costo presentan limitaciones que dificultan su adopción en empresas pequeñas y medianas. Muchos de estos equipos requieren conexión permanente a internet, operan con software propietario o solo permiten exportar datos en formatos estáticos como CSV, lo que genera dependencia de procesos manuales y restringe la interoperabilidad con otros sistemas.

Al mismo tiempo, la mayoría de los ERP utilizados por las empresas no incorpora módulos nativos para capturar asistencia desde hardware, por lo que su integración con dispositivos externos resulta costosa o directamente incompatible. La ausencia de APIs abiertas, la falta de mecanismos de sincronización automática y la poca flexibilidad para adaptar el hardware al contexto operativo dificultan la incorporación de tecnología biométrica en entornos reales.

Estas limitaciones se vuelven más evidentes en escenarios donde la conectividad es inestable. Muchos relojes de control no pueden almacenar marcaciones localmente de manera segura ni realizar una sincronización diferida una vez restablecida la red, lo que expone a las empresas a la pérdida de datos y al incumplimiento normativo. asimismo, la mayoría de las soluciones comerciales no permite personalizar la lógica de validación biométrica, integrar módulos anti fraude ni extender el sistema a necesidades particulares, como gestión de turnos o funcionamiento híbrido.

Actualmente no existe en el mercado una alternativa accesible, modular y adaptable que permita capturar asistencia mediante biometría (huella o reconocimiento facial), operar de forma autónoma sin internet, almacenar registros en el dispositivo, detectar intentos de suplantación y sincronizar datos con sistemas ERP mediante APIs estándar. Esta brecha tecnológica afecta especialmente a organizaciones que requieren una solución flexible, de bajo costo y consistente con las exigencias de la autoridad laboral.

En síntesis, el problema consiste en la falta de un reloj control biométrico capaz de operar offline, almacenar y sincronizar marcaciones de manera segura, y conectarse a

sistemas ERP mediante interfaces estándar cumpliendo la normativa vigente.

1.3 Propuesta de solución

La propuesta consiste en diseñar e implementar un prototipo de control de asistencia utilizando un módulo ESP32-CAM junto con un lector de huellas dactilares. El ESP32-CAM es una placa de bajo costo que incorpora una cámara, conectividad Wi-Fi y Bluetooth, lo que permite capturar imágenes, procesarlas y comunicarse con otros sistemas sin necesidad de equipos adicionales.

El objetivo es que el dispositivo pueda registrar la asistencia de los trabajadores mediante reconocimiento facial y huella digital, funcionando tanto con conexión a internet como sin ella. En modo offline, el equipo almacenará temporalmente los registros en una tarjeta microSD o en su memoria interna, lo que le permitirá operar de manera autónoma durante varios días o incluso semanas, dependiendo de la capacidad de almacenamiento configurada. Cuando se recupere la conexión, los datos se sincronizarán automáticamente con un servidor web o con un sistema ERP, según la configuración establecida por el usuario.

El lector de huellas permitirá complementar la verificación facial, ofreciendo una alternativa en lugares donde las condiciones de luz o el ángulo de la cámara dificulten el reconocimiento.

El prototipo contará además con una interfaz web integrada, accesible desde la misma red local, que permitirá configurar parámetros de red, administrar usuarios y definir el destino de los datos. Junto con ello, se desarrollará una documentación técnica de apoyo, donde se describirá el funcionamiento general del sistema, el formato de los registros generados y las opciones de integración con diferentes plataformas ERP. Esta integración se realizará mediante un endpoint o API estándar, lo que permitirá la conexión con cualquier sistema ERP que acepte intercambio de datos a través de solicitudes HTTP o JSON, brindando flexibilidad sin depender de un proveedor específico.

Finalmente, para mejorar la seguridad en el proceso de marcación, se incorporarán métodos que eviten la suplantación de identidad, impidiendo que se utilicen fotografías o imágenes impresas para registrar asistencias falsas.

1.4 Objetivos

En el siguiente apartado se procederá a presentar el objetivo general del proyecto, junto con los objetivos específicos.

1.4.1 Objetivo general

Probar la utilidad de un sistema de control de asistencia IoT de bajo costo y código abierto, desarrollado con un ESP32-CAM, que permita optimizar la gestión de asistencia laboral en pequeñas y medianas empresas mediante integración con ERP y funcionamiento offline.

1.4.2 Objetivos específicos

Objetivo específico 1

Diseñar la arquitectura general del sistema IoT, definiendo componentes y flujos de datos.

Objetivo específico 2

Implementar el módulo biométrico de registro de asistencia mediante reconocimiento facial y huella digital.

Objetivo específico 3

Incorporar capacidad de funcionamiento offline con almacenamiento local y sincronización posterior.

Objetivo específico 4

Desarrollar una interfaz web embebida para la configuración y administración del dispositivo.

Objetivo específico 5

Implementar una API de comunicación para la integración con servidores o sistemas ERP externos.

Objetivo específico 6

Integrar y validar el funcionamiento conjunto del hardware y software desarrollado.

Objetivo específico 7

Evaluar el desempeño, confiabilidad y usabilidad del sistema en escenarios de prueba reales.

Objetivo específico 8

Analizar los resultados obtenidos para determinar la viabilidad.

1.5 Alcances

A continuación se presentarán los alcances propuestos para este proyecto.

- Se diseñará la arquitectura de hardware y software del sistema, incluyendo los módulos de captura de imagen, registro de huella, almacenamiento local y comunicación con el servidor.
- Se implementará un prototipo funcional capaz de registrar asistencias mediante reconocimiento facial y huella digital, operando en modo offline con almacenamiento local y sincronización posterior.
- Se desarrollará una interfaz web embebida para la configuración del dispositivo, administración de usuarios y definición del destino de los datos.
- Se integrará una API RESTful para permitir la comunicación entre el dispositivo y un servidor o sistema ERP, utilizando formatos de intercambio JSON sobre HTTP.
- Se incorporarán medidas básicas de seguridad biométrica orientadas a evitar la suplantación de identidad mediante imágenes o fotografías.
- Se realizarán pruebas de funcionamiento offline, midiendo la autonomía del sistema (tiempo máximo de operación sin conexión y capacidad de almacenamiento).
- Se ejecutarán pruebas de integración entre los módulos de hardware, software y el ERP simulado, verificando el flujo completo de datos desde el registro hasta la sincronización.
- Se aplicarán pruebas de usabilidad para evaluar la facilidad de configuración y el nivel de satisfacción del usuario final con la interfaz embebida.
- Se efectuarán pruebas de confiabilidad e integridad, comprobando que los datos registrados sin conexión se transmitan correctamente una vez restablecida la conectividad.
- Los resultados se analizarán y documentarán para determinar la viabilidad técnica y operativa del sistema propuesto.

- El prototipo hará uso de la conexión a internet mediante Wi-Fi para realizar las sincronizaciones con los datos existentes en el servidor como así las funcionalidades de reconocimiento facial avanzado.

1.6 Resumen capítulo

El capítulo 1 presenta el contexto normativo del control de asistencia en Chile y expone las limitaciones de los sistemas biométricos actuales, especialmente su dependencia de internet y su baja interoperabilidad con ERP. Ante esta problemática, se propone un sistema IoT basado en ESP32-CAM con autenticación por rostro y huella, capaz de operar offline y sincronizar datos posteriormente. Además, se establecen los objetivos, alcances y lineamientos generales del proyecto.

2. Marco teórico

A continuación se presentan los fundamentos teóricos y tecnológico que facilitan la comprensión del proyecto

2.1 Conceptos básicos

2.1.1 Conceptos globales

- Control de asistencia: Proceso por el cual una empresa u organización registra las horas de entrada y salida de sus trabajadores.
- Sistema ERP: EntERPrise Resource Planning, sistema de gestión empresarial que permite procesos de una empresa u organización, tales como recursos humanos, contabilidad, finanzas, inventario, asistencia.
- Internet de las cosas (IoT): Paradigma tecnologico que permite la conexión entre dispositivos físico a través de redes de comunicación.
- Resolucion exenta n°38: Norma promulgada por la dirección del trabajo (DT) en 2024 (actualizada en 2025), que establece los requisitos obligatorios para los sistemas electrónicos de registro y control de asistencia.
- Biometria: La biometría corresponde al conjunto de técnicas utilizadas para identificar personas mediante características físicas o conductuales únicas, tales como el rostro o la huella digital. Este tipo de autenticación es ampliamente utilizada en sistemas de control de asistencia por su alta precisión y dificultad de suplantación [36].
- Sincronización offline-online: Proceso mediante el cual un dispositivo opera sin conexión utilizando almacenamiento local y, una vez restablecida la conectividad, transmite los datos acumulados hacia un servidor o ERP. Resulta esencial en entornos con conectividad inestable.

2.1.2 Conceptos de software

- ESP32-CAM: Microcontrolador basado en el esp32, con la inclusión de una cámara OV2640, conectividad WiFi y Bluetooth, además de contar con la capacidad de procesamiento local. [6]
- API (Aplicacion Programming Interface): Conjunto de métodos o puntos de conexión que permiten la conexión entre diferentes aplicaciones mediante protocolos de redes. [7]
- Backend: Parte del sistema encargada de la lógica del negocio, manejo de datos y comunicación con bases de datos. [8]
- Frontend: Conjunto de interfaces y funcionalidades con las que interactúa el usuario final. [9]
- Mqtt: Protocolo de comunicación ligero basado en el modelo publicador/suscriptor, diseñado para intercambiar mensajes de manera eficiente, utilizado entre dispositivos que cuenten con recursos limitados.[10]
- Javascript Object Notation (JSON): Formato de datos que se suele utilizar en entornos de desarrollo web para el intercambio de datos de manera legible tanto por las personas como las maquinas. [11]
- SPIFFS (SPI Flash File System): Sistema de archivos diseñado para microcontroladores que permite almacenar información en memoria flash mediante archivos. En este proyecto se utiliza para guardar usuarios, turnos, asignaciones y asistencias en formato JSON, permitiendo el funcionamiento offline [37].
- API RESTful: Estilo de arquitectura de servicios web basado en el protocolo HTTP, que permite la comunicación entre aplicaciones usando operaciones estándar como GET, POST, PUT o DELETE. Es utilizado para integrar el dispositivo IoT con sistemas ERP externos mediante intercambio de datos en formato JSON [38].
- Microservicios: Arquitectura de software basada en dividir un sistema en servicios independientes que se comunican entre sí. Esta estructura permite escalabilidad y facilita la separación entre el procesamiento de imágenes, backend y comunicación con ERP[39].
- Broker MQTT: Servidor encargado de recibir, gestionar y distribuir mensajes enviados por dispositivos IoT utilizando el protocolo MQTT. Actúa como intermediario entre el publicador (el ESP32-CAM) y los suscriptores (backend u otros servicios)[40].

- Sensor PIR (Passive Infrared Sensor): Sensor que detecta cambios en la radiación infrarroja del entorno para identificar la presencia de personas por su calor corporal. En este proyecto se utiliza como “portero” del sistema: activa los mecanismos de captura biométrica solo cuando detecta movimiento frente al dispositivo, reduciendo el consumo energético y evitando falsas activaciones. A diferencia de los sensores IR activos, el PIR no emite luz infrarroja sino que detecta pasivamente la que irradian los cuerpos.
- Anti-spoofing biométrico: Técnica de seguridad que permite detectar intentos de suplantación de identidad mediante fotografías, pantallas o máscaras frente a un sistema de reconocimiento biométrico. DeepFace, la librería utilizada en este proyecto, incorpora un detector de suplantación basado en redes neuronales convolucionales que analiza micro-texturas, reflectancia de la piel y profundidad en la imagen para distinguir entre un rostro real y una superficie plana.
- JWT (JSON Web Token): Estándar abierto (RFC 7519) que define un formato compacto para transmitir información de autenticación y autorización entre partes como un objeto JSON firmado digitalmente. En este proyecto se utiliza para proteger los endpoints del backend, transportando el identificador del usuario, su empresa y su rol, con firma HMAC-SHA256 y expiración de 24 horas.
- Arquitectura multi-tenant: Modelo de software donde una única instancia de la aplicación atiende a múltiples organizaciones (empresas), manteniendo los datos de cada una lógicamente aislados. En este proyecto se implementa mediante una columna `empresa_id` en las tablas principales del sistema, garantizando que los usuarios de una empresa no puedan acceder a los datos de otra.
- Cifrado Fernet: Esquema de cifrado simétrico que utiliza AES-128 en modo CBC con autenticación HMAC-SHA256, proporcionando confidencialidad e integridad. En este proyecto se emplea para cifrar los embeddings faciales (vectores de características biométricas) antes de almacenarlos en la base de datos, protegiendo los datos sensibles incluso ante accesos no autorizados a la base de datos.
- DeepFace: Framework de reconocimiento facial de código abierto que integra múltiples modelos de deep learning (Facenet, VGG-Face, ArcFace, entre otros) y detectores faciales (RetinaFace, MTCNN, OpenCV). En este proyecto se utiliza con el modelo Facenet para generar embeddings faciales de 128 dimensiones y con el detector RetinaFace para localizar rostros en las imágenes capturadas por el ESP32-CAM.

2.2 Tecnologías utilizadas

A continuación se presentarán todas las tecnologías que harán presente durante el desarrollo del proyecto. Dentro de estas

2.2.1 Microcontrolador ESP32-CAM

El ESP32-CAM es un dispositivo de desarrollo que integra un microcontrolador ESP32-CAM con conectividad Wi-Fi y Bluetooth, además de una cámara OV2640. La elección de este dispositivo se debe a su bajo costo, capacidad de procesamiento local, compatibilidad con el entorno de Arduino y su amplia comunidad de soporte. De las características que se pueden destacar encontramos la captura de imágenes para el reconocimiento facial, gestionar sensores biométricos y mantener conectividad directa con el servidor o la red local. Otras alternativas como la Raspberry Pi Zero W o Arduino MKR1000 fueron descartadas por su mayor consumo y costo implementación. [12]

2.2.2 Lector de huella digital AS608

El lector de huellas digital usa el sensor AS608, un lector de huella con la capacidad de registrar y verificar la identidad de los usuarios mediante patrones dactilares. La elección de este dispositivo fue por la compatibilidad con el ESP32-CAM, su bajo consumo y su resistencia a condiciones de luz ambiental en comparación a otros sensores ópticos tradicionales como el R307. El AS608 se comunica con el microcontrolador a través de una interfaz UART, y la integración de este se realiza mediante la biblioteca Adafruit FingERPint Sensor Library, lo que facilita el registro, búsqueda y verificación de huellas directamente desde el dispositivo IoT. [13]

2.2.3 Protocolos de comunicación HTTP

El protocolo de comunicación HTTP (Hypertext Transfer Protocol) es un protocolo cliente-servidor utilizado para transmitir información mediante solicitudes y respuestas que se transmite a través de los métodos GET, POST, PUT Y DELETE, generalmente utilizando formatos estructurados como JSON (JavaScript Object Notation). Permite la comunicación directa entre dispositivos y servidores, por lo que para este proyecto se le puede dar un buen uso para realizar la petición de información.[14]

2.2.4 Protocolo de comunicación MQTT

El protocolo de comunicación MQTT o Message Queuing Telemetry Transport es un protocolo ligero de mensajería diseñado específicamente para entornos IoT. Se base en la arquitectura de publicador-suscriptor, donde los dispositivos envían mensajes a un broker central que los distribuye entre los clientes suscritos a los distintos tópicos que pueden existir. Este modelo reduce el uso de ancho de banda, lo que lo hace ideal para redes con conectividad limitada o intermitente. [15]

2.2.5 Contenedores Docker

Docker es una plataforma de virtualización ligera que permite empaquetar aplicaciones junto las dependencias dentro de servidores virtuales aislados denominados contenedores. Cada contenedor ejecuta un entorno idéntico al que se utilizaría en producción, lo que facilita el despliegue, la portabilidad de los sistemas. Para el área de IoT y web, Docker es útil para ejecutar servicios como bases de datos, servidores o brokers MQTT de forma independiente y controlada, lo que lo hace útil para el trabajo al momento de levantar los servicios en distintos servidores. [16]

2.2.6 Sistemas de bases de datos

Un sistema de gestor de bases de datos es un conjunto de programas que permiten administrar bases de datos de manera estructurada. La principal función es proporcionar una interfaz entre los usuarios y los datos almacenados, garantizando integridad consistencia y seguridad de la información. Los SGBD relacionales, basado en el modelo propuesto por Edgar F. Codd (1970), organizan los datos en tablas vinculadas mediante relaciones, lo que facilita su consulta a través del lenguaje SQL. Lo SGBD más utilizados que se destacan son MySql, MariaDb, Oracle Database y PostgreSQL. Estos sistemas ofrecen las características fundamentales, tales como el control de concurrencia, mecanismos de respaldo y recuperación, gestión de usuarios y soporte para transacciones atómicas (ACID). PostgreSQL es un sistema de base de datos relacional y orientado a objetos de código abierto, desarrollado inicialmente en la Universidad de California en Berkeley, caracterizada principalmente por ofrece un motor avanzado con soporte para consultas complejas, procedimientos almacenados, tipos de datos personalizados, índices avanzados y compatibilidad con JSON y XML, lo que permite la fácil integración con aplicaciones web e IoT. [17] Para el contexto del proyecto PostgreSQL se utiliza como base de datos, ya que tiene compatibilidad con entornos de desarrollo como python.

2.2.7 Sensor PIR (HC-SR501)

El sensor infrarrojo pasivo HC-SR501 es un módulo de bajo costo diseñado para detectar movimiento mediante la medición de cambios en la radiación infrarroja del entorno. Cuando una persona se sitúa dentro de su campo de visión (aproximadamente 6 metros de alcance y 120° de apertura), el sensor emite una señal digital HIGH. En este proyecto se utiliza como detector de presencia para activar los mecanismos de captura biométrica solo cuando hay una persona frente al dispositivo, optimizando así el consumo energético del lector de huellas AS608 y la cámara OV2640 durante los períodos de inactividad.

La elección del sensor PIR, en lugar del sensor IR activo originalmente planificado, respondió a dos factores: (1) la extrema limitación de pines GPIO disponibles en el ESP32-CAM tras la asignación del bus paralelo de la cámara, que hubiera requerido sacrificar otra funcionalidad para añadir el sensor IR; y (2) la disponibilidad del detector anti-spoofing integrado en DeepFace, que proporciona una capa de seguridad contra suplantación mediante software sin necesidad de hardware adicional [34][35].

2.2.8 Framework DeepFace

DeepFace es un framework de reconocimiento facial para Python desarrollado por Sefik Ilkin Serengil, que envuelve múltiples modelos de deep learning en una API unificada. Soporta los modelos Facenet, VGG-Face, ArcFace, DeepFace, DeepID y Dlib, así como los detectores faciales RetinaFace, MTCNN, OpenCV, SSD y Dlib. Para este proyecto se seleccionó el modelo Facenet por su equilibrio entre precisión y tamaño de embedding (128 dimensiones), y el detector RetinaFace por su robustez ante variaciones de pose, iluminación y oclusión parcial.

DeepFace incluye además un módulo de anti-spoofing capaz de distinguir entre rostros reales y fotografías o pantallas, analizando micro-texturas y reflectancia. Esta funcionalidad, combinada con el sensor PIR, constituye la estrategia antifraude del prototipo sin requerir hardware especializado adicional.

2.2.9 Broker Mosquitto en contenedor Docker

Eclipse Mosquitto es un broker MQTT de código abierto, ligero y adecuado para dispositivos de bajo consumo. En este proyecto se despliega mediante Docker Compose, utilizando la imagen oficial `eclipse-mosquitto:latest`, exponiendo el puerto 1883 para conexiones MQTT nativas y el puerto 9001 para WebSockets. Esta configuración permite que el ESP32-CAM pueda conectarse al broker tanto mediante MQTT sobre TCP como mediante WebSockets seguros (`wss://`), facilitando la comunicación en entornos con firewalls o proxies que bloquean puertos no estándar [16][41].

2.3 Estado del arte

A continuación se revisaran los sistemas y tecnológicas existentes relacionados dispositivos de control de asistencia biométricos y su integración con plataformas empresariales, con el propósito de identificar el estado que se encuentra tanto el mercado actual como los avances más relevantes e indicar las limitaciones que motivan el desarrollo de la propuesta de este proyecto.

Si bien los sistemas de control de asistencia es un campo del cual se tienen variados recursos y sistemas, la gran mayoría se encuentran ligados a un software propio, restringiendo a los sistemas ERP tener que pagar suscripciones a estos sistemas.

2.3.1 Soluciones basadas en hardware

En esta sección hará énfasis en aquellos dispositivos de reloj control biométricos que permiten registrar la asistencia de los trabajadores mediante la verificación de características únicas, como huellas digitales, reconocimiento facial, entre otras. Estos equipos incluyen funcionalidad tales como almacenamiento interno, conectividad por red (LAN/Wi-Fi), a continuación se describirán los principales dispositivos utilizados en Chile.

1. GeoVictoria, relojes de control: GeoVictoria dentro de sus planes y productos se encuentran hardware biométrica con una plataforma en la nube para realizar el control de asistencia. Estos dispositivos pueden operar mediante una conexión LAN o 3G para zonas remotas. Sin embargo dentro de sus limitantes encontramos que su API esta sujeto a restricciones comerciales y acceso mediante suscripción, lo que limita la personalización o integración libre. Además se tiene un ecosistema cerrado que dificulta modificaciones independientes por parte del cliente [18].
2. ZKTeco, terminales biométricas de asistencia: ZKTeco es una marca internacional con presencia en Chile y Latinoamérica en general, esta compañía ofrece terminales de control de asistencia "stand-alone" que combinan distintos métodos de registro tales como huella, rostro, PIN y tarjeta. Su limitante es que se encuentra cerrado totalmente a su sistema o quedan sin integración directa con un ERP, a su vez que ofrece exportación de datos mediante archivos csv siendo poco amigable para su uso en un ERP [19].
3. Dispositivos genéricos: Consideramos como dispositivos genéricos cualquier tipo de reloj control que se pueda adquirir en algún portal de venta web, que incluyan características tales como marcaje mediante huella o rostro. En base a lo anterior la gran mayoría de estos dispositivos tienen la capacidad de aplicar estas funciones

pero al costo que su información solo queda en el dispositivo, teniendo métodos como exportacion de datos por usb en un formato que solo se pueden visualizar y difícil de integrar a ERP.

2.3.2 Soluciones basadas en software

En el siguiente apartado se presentarán soluciones relacionadas al control de asistencia relacionados con plataformas web o servicios en la nube (Saas) que permiten gestionar la asistencia laboral sin requerir un reloj biométrico físico.

1. Talana/Buk: Talana y Buk son plataformas chilenas que ofrecen módulos de gestión de asistencia, turnos, entre otras características mediante aplicaciones web, en ambas aplicaciones se encuentran en la misma situación, con dependencia de conectividad a internet, ausencia de hardware biométrico físico, y costos asociados a suscripciones web [20][21].
2. Defontana/Softland: Defontana y Softland son empresas dedicadas a implementar sistemas ERP para las empresas, dentro de sus funcionalidades integran control de asistencia, permitiendo registrar las jornadas de trabajo, cálculo de horas y conexión con módulo de remuneraciones. La gran limitante de estas aplicaciones web que están ligadas 100% a su uso web, es decir, se debe contratar el ERP completo [22][23].

De los productos y servicios presentados anteriormente podemos concluir que el mercado chileno cuenta con una oferta amplia en cuanto a soluciones para el control de asistencia, tanto en dispositivos biométricos, como en plataformas en la nube mediante software. Sin embargo cada uno de los elementos presentados en este estudio del arte, presentan limitaciones en cuanto a su estructura.

Por un lado tenemos los relojes de control biométricos ofrecidos por GeoVictoria o ZKTeco, que tienen soluciones bastante adaptadas a las necesidades del usuario pero con un enfoque en empresas más grandes, ligados a software propietario de una conectividad constante con los servidores del fabricante, esto reduciendo la flexibilidad, elevando costos para los servicios que los ERP finalmente ofrecieran a las medias y pequeñas empresas.

Por otra parte las soluciones basadas en software o servicios SaaS, tales como Talana, Buk, Defontana y Softland se centran en la gestión de la asistencia de manera remota y mediante plataformas web, limitándose en su ambiente operacional dedicado solo para trabajar sobre los mismos servicios de la misma empresa, además de haber ausencia de hardware, lo que limita la aplicabilidad en entornos con condiciones poco ideales para el funcionamiento con internet constante.

En consecuencia de lo comentado anteriormente, se evidencia una falta de soluciones híbridas que puedan unir elementos como la autenticación biométrica, con la integración configurable hacia sistemas ERP, sin depender totalmente de una infraestructura externa.

2.4 Metodologías

En este apartado se presentan las metodologías de desarrollo y evaluación que serán utilizadas para este proyecto.

2.4.1 Metodología de desarrollo

Considerando que el proyecto está relacionado con el desarrollo de un prototipo de IoT, y pensando en su futura implementación en un entorno real, se identificaron distintos tipos de metodologías de desarrollo que permitan desarrollar el prototipo de manera que cada iteración, ciclo, o tarea sea una funcionalidad nueva que se adhiera al prototipo. Dado lo anterior se ha realizado una búsqueda para identificar la metodología que más acorde esté para este proyecto. Dentro de las metodologías que si encajaban, dado el contexto del proyecto se encontraron las siguientes metodologías: Scrum, y el modelo de prototipado evolutivo incremental. Dado que Scrum requiere de más actores que estén presentes en el proyecto y al adaptarla para un trabajo en solitario se perderían elementos de esta metodología tales como los roles, como así la revisión colaborativa que son parte de la estructura de la metodología[24]. Es por esta razón se descarto esta metodología y se optó por emplear una metodología de prototipado evolutivo incremental adaptándola a un proyecto en solitario, es decir, se reemplaza lo que sería el cliente, que es quien solicita los requisitos, por el profesor guía quien guiará y entregará retroalimentación de las iteraciones. Esta metodología se adecuaba para este proyecto al tratarse de una metodología más apta y preparada para lo que es el desarrollo de prototipos y de mejoras e integraciones continuas[25][26].

La metodología de prototipado evolutivo con un enfoque incremental propone un desarrollo basado en versiones sucesivas del prototipo, en la que cada iteración incorpora nuevas funcionalidades y mejoras sobre el anterior, de esta forma se puede probar, evaluar y ajustar continuamente el sistema a medida que va evolucionando. La estructura de la metodología es la siguiente:

- **Diseño y planificación:** Se definen los objetivos específicos de cada iteración, se seleccionan componentes y se diseña la arquitectura.
- **Implementación y prueba:** Se desarrolla la funcionalidad planificada, se integra con los módulos que ya existen y se verifica su correcto funcionamiento

- Evaluación y mejora: Una vez desarrollado los procesos anteriores se analizan los resultados obtenidos identificación errores y planificando ajustes para la siguiente iteración.

[27]

Cada una de estas fases termina con un incremento funcional validado, lo que permite mantener un progreso controlado y documentado del prototipo. A continuación se presenta un diagrama en la cual representamos la metodología propuesta.

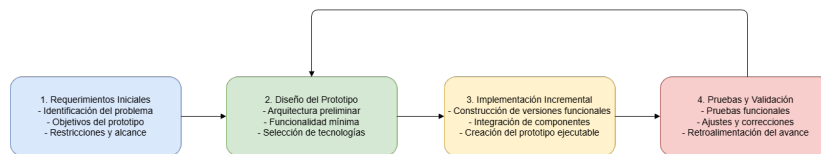


Figura 2.1: Metodología prototipado evolutivo con enfoque incremental

2.4.2 Metodología de evaluación

Para la evaluación del proyecto se deben considerar 3 aspectos claves para determinar la viabilidad del proyecto, de los cuales nos permiten evaluar y verificar nuestros objetivo. Las pruebas evaluarán el prototipo, enfocandose en como funciona con todas las piezas y/o software en conjunto, saber que tan útil puede llegar a ser, y finalmente al tener como objetivo su uso en un ERP, se necesita saber el nivel de satisfacción que se tiene por el prototipo final.

Pruebas de integración

Las pruebas de integración permiten evaluar diferentes componentes de un sistema ya sea hardware software o base de datos, asegurando que todos estos componentes funcionen correctamente en conjunto asegurando la comunicación e interoperabilidad entre ellos. Para este caso se realizarán pruebas de integración ascendentes de manera que se vayan realizando pruebas de los módulos más pequeños hasta llegar al módulo más grande [28]. La selección de esta prueba va relacionada con la metodología de desarrollo que consite en el prototipado evolutivo, el cual se integran progresivamente los módulos validos en etapas previas.

Pruebas de usabilidad

Las pruebas de usabilidad tienen como objetivo evaluar la facilidad de uso, comprensión y satisfacción del usuario con el sistema, conforme a los estándares de calidad de software [29] [30]. Para llevar a cabo las pruebas de usabilidad se utilizará el System Usability Scale (SUS) [31]. El SUS es un método reconocido por su simplicidad de aplicación

y por permitir una evaluación rápida y fiable de la usabilidad de un sistema [32]. La metodología del SUS consiste en obtener una puntuación global de usabilidad mediante la aplicación de un cuestionario basado en una escala de Likert de 5 puntos, compuesto por 10 preguntas que abordan diferentes aspectos de la experiencia de usuario, como la facilidad de uso, la claridad de la interfaz y la eficacia del sistema [31].

La razón del uso de esta prueba se debe a que el prototipo propuesto "prototipo de Control de asistencia Biometrico", debe ser fácil de usar para los trabajadores puedan realizar marcaciones sin complicaciones, como así para quienes integran este sistema a su ERP.

Pruebas de confiabilidad y disponibilidad

Las pruebas de confiabilidad y disponibilidad son cruciales en sistemas que operan en entornos con conectividad intermitente o hardware limitado, como los dispositivos IoT[33]. La finalidad de la prueba de confiabilidad es medir la capacidad del sistema para funcionar correctamente bajo condiciones normales durante un periodo prolongado sin errores ni pérdidas de datos, mientras que las pruebas de disponibilidad hace referencia a la capacidad del sistema para mantenerse operativo incluso cuando se presentan condiciones extremas, tales como pérdidas conexión, desconexión temporal de energía[34], etc. P Según Perry & Petry, estas pruebas son esenciales para garantizar que el sistema mantenga la integridad de los datos y su disponibilidad en todo momento[35].

2.5 Resumen del capítulo

El capítulo 2 presenta el marco teórico que sustenta el proyecto, describiendo los conceptos esenciales relacionados con control de asistencia, IoT, ERP, y los requisitos normativos definidos por la Resolución Exenta N.º38. Luego se detallan las tecnologías utilizadas, incluyendo el microcontrolador ESP32-CAM, el lector de huellas AS608, los protocolos HTTP y MQTT, el uso de contenedores Docker y la base de datos PostgreSQL.

También se revisa el estado del arte, analizando las limitaciones de dispositivos biométricos comerciales y plataformas SaaS como Talana, Buk, Defontana y Softland, evidenciando la falta de soluciones híbridas, abiertas y con funcionamiento offline. Finalmente, el capítulo expone las metodologías de desarrollo y evaluación seleccionadas, donde se justifica el uso del prototipado evolutivo incremental y se describen las pruebas de integración, usabilidad y confiabilidad que se aplicarán al sistema.

3. Metodologías

En el presente capítulo esta destinado a describir la manera en la que se aplicarán las metodologías propuestas en el capítulo anterior, además de presentar los resultados esperados.

3.1 Metodologia de desarrollo

Como se menciona el capítulo anterior en la sección 2.4.1 el proyecto propuesto se lleva a cabo mediante una metodología de tipo prototipado evolutivo con enfoque incremental. Esta metodología permite crear versiones funcionales del prototipo de manera progresiva, mejorando en cada iteración en base a pruebas funcionales del prototipo.

A continuación se procederá a explicar las distintas fases del proceso y como se aplicará la metodología.

3.1.1 Protipado Evolutivo Incremental

La metodología de prototipado evolutivo incremental se aplica en este proyecto como un enfoque de desarrollo iterativo cuyo objetivo principal es construir el prototipo del sistema de manera progresiva, mediante versiones funcionales que incorporan nuevas capacidades en cada iteración. A diferencia de modelos secuenciales como el modelo en cascada, este enfoque permite validar tempranamente el funcionamiento real del dispositivo IoT, ajustar sus componentes según las pruebas realizadas y asegurar que cada incremento aporte mejoras concretas sobre el prototipo anterior.

Esta metodología se adecuía debido a que la propuesta de proyecto combina hardware (ESP32-CAM, lector de huella AS608 y sensor PIR), software embebido, almacenamiento local, backend, mecanismos biométricos y una interfaz web. Dado que estos elementos funcionan como módulos independientes que deben integrarse correctamente, un modelo basado en incrementos funcionales permite validar cada módulo antes de avanzar al siguiente nivel de complejidad.

El proceso de prototipado evolutivo incremental se aplica siguiendo tres fases principales: diseño y planificación, implementación y prueba, y evaluación y mejora, las cuales se repiten en cada iteración del proyecto. A continuación, se explica cómo se aplican estas fases dentro del desarrollo del prototipo.

1. Fase de diseño y planificación

En la fase de diseño y planificación se definen los objetivos específicos de la iteración correspondiente, así como los módulos que se incorporarán al prototipo. Para cada ciclo se identifican los componentes de hardware y software involucrados, los flujos de datos necesarios y la arquitectura parcial que debe soportar la nueva funcionalidad.

Durante esta fase se desarrollan diagramas preliminares, modelos de datos, mockups y definiciones de comunicación entre módulos. Asimismo, se determinan las dependencias entre componentes, lo que permite asegurar que el incremento se pueda integrar adecuadamente con los elementos ya existentes. Esta etapa garantiza que cada iteración tenga un alcance claro, siguiendo un orden lógico de complejidad creciente.

2. Fase de implementación y prueba

En esta fase se procede a construir la funcionalidad definida en el diseño de la iteración. La implementación incluye el desarrollo del firmware del ESP32-CAM, la programación del lector de huellas AS608, la gestión del almacenamiento en SPIFFS, el desarrollo del backend o la implementación de los modelos biométricos, según corresponda al incremento.

Paralelamente, se realizan pruebas integradas para verificar que la nueva funcionalidad opere correctamente en conjunto con las versiones anteriores del prototipo. Estas pruebas incluyen verificaciones de comunicación entre módulos, consistencia de datos, estabilidad del firmware, validación de imágenes capturadas, o evaluación de la sincronización con el servidor.

El objetivo es garantizar que el incremento sea plenamente funcional antes de ser incorporado como parte estable del prototipo.

3. Fase de evaluación y mejora

Finalizada la implementación, cada incremento es sometido a una fase de evaluación donde se analizan los resultados obtenidos. En esta etapa se identifican errores, limitaciones y posibles mejoras, considerando el funcionamiento real del dispositivo tanto en escenarios conectados como offline.

Cuando corresponde, se realizan ajustes al diseño, se corrigen fallas detectadas y se optimiza el desempeño de los componentes integrados. Esta fase también permite

preparar la entrada para la siguiente iteración, asegurando que el prototipo evolucione de manera controlada y mantenga coherencia con los objetivos del proyecto.

3.2 Planificación

La planificación del proyecto se estructura mediante la metodología de prototipado evolutivo con enfoque incremental. Cada iteración incorpora nuevas funcionalidades sobre el prototipo anterior, permitiendo evaluar, corregir y fortalecer el sistema progresivamente. A continuación, se detalla cada iteración, incluyendo sus fases de análisis, diseño, implementación y pruebas.

Con el fin de poder contrarrestar el avance del desarrollo del proyecto cada iteración corresponde a un 12,5% del total del proyecto, donde además cada porcentaje de tarea corresponde a la división de 12,5% entre la cantidad de tareas a realizar.

3.2.1 Iteración 1: Integración de hardware y servidor embebido

Análisis

- Identificación de requisitos base: captura facial mediante cámara OV2640, lectura de huella con sensor AS608 y servidor web local.
- Definición de restricciones del proyecto: memoria RAM del ESP32 (520 KB), escasez de GPIO tras asignación del bus de cámara (uso forzado de GPIO14/15 para UART), consumo del flash LED (250 mA).
- Selección preliminar de arquitectura del sistema IoT con soporte para operación autónoma vía Access Point (SSID: ESP32-ASISTENCIA, pass: Asistencia2026).

Diseño

- Diseño de la arquitectura física (microservicios con broker MQTT) y lógica (módulos de firmware del ESP32-CAM).
- Diseño de mockups base para la interfaz del dispositivo: menú principal, configuración Wi-Fi, registro de usuario, lista de usuarios, turnos, asignaciones.
- Diseño del flujo operacional inicial de marcación con y sin conectividad.

Implementación

- Configuración inicial del ESP32-CAM: cámara OV2640 en VGA JPEG (calidad 8), flash LED controlado por PWM (5 kHz, 50% duty en GPIO4) para evitar caídas de tensión.

- Programación del lector AS608 mediante UART secundaria (GPIO14=RX, GPIO15=TX, 57600 baudios), inicialización con verificación de contraseña.
- Calibración del sensor PIR (GPIO12, pull-down) como detector de presencia con tiempo de estabilización de 3 segundos.
- Implementación de servidor HTTP en puerto 80 con 9 rutas web y 14 endpoints de acción (handlers), más AP fallback automático (192.168.4.1).
- Construcción de vistas HTML embebidas con literales `R"rawliteral(...)"` sin dependencias CDN externas.

Pruebas

- Prueba de captura de imagen JPEG continua desde la cámara OV2640.
- Prueba de comunicación UART y verificación de contraseña del AS608.
- Prueba de enrolamiento de huella (detección, conversión a plantilla, almacenamiento en slot).
- Prueba de carga de vistas HTML desde el AP (192.168.4.1) y desde Wi-Fi externa.

3.2.2 Iteración 2: Almacenamiento local y modo offline

Análisis

- Selección del formato de almacenamiento: archivos JSON mediante LittleFS (sucesor de SPIFFS para ESP32).
- Identificación de parámetros configurables por el usuario: Wi-Fi (SSID, pass), backend URL, broker MQTT, PIN de enrolamiento, ERP config.
- Requisito de persistencia tras reinicios y de compatibilidad con futura sincronización (campo `sincronizado` booleano en cada entidad).

Diseño

- Diseño de 6 estructuras JSON: `personas.json`, `turnos.json`, `asignaciones.json`, `asistencias.json`, `wifi.json`, `erp-config.json`.
- Diseño de la lógica de alternancia automática entrada/salida y validación de turno activo (`turnoActivo()`).

Implementación

- Implementación de funciones genéricas `loadArray()` y `saveArray()` para lectura/escritura de archivos JSON en LittleFS.
- Implementación del flujo de registro de huella en 3 estados (captura inicial, espera de retiro, segunda captura y verificación), con búsqueda de slot libre (1–127).
- Implementación de marcación offline: lectura de huella → identificación → validación de turno → alternancia entrada/salida → persistencia con `sincronizado = false`.
- Implementación de gestión completa offline: CRUD de turnos, asignaciones, consulta de asistencias, todo desde la interfaz web embebida.

Pruebas

- Prueba de persistencia de datos tras reinicios del ESP32-CAM.
- Prueba de lectura y deserialización correcta de los 5 archivos JSON.
- Prueba de escritura concurrente sin corrupción de archivos.
- Prueba de marcación offline por huella con validación de turno y alternancia entrada/salida.
- Prueba de funcionamiento del AP sin conectividad externa.

3.2.3 Iteración 3: Backend, base de datos y comunicación

Análisis

- Evaluación del uso de HTTP (para API REST y sincronización) y MQTT (para transmisión de imágenes faciales en tiempo real).
- Análisis de la conectividad vía Wi-Fi con mecanismo de reconexión progresiva (backoff 3–15 segundos, 5 intentos máximos) y watchdog periódico.
- Evaluación de PostgreSQL como base de datos relacional, con Docker Compose para el broker Mosquitto y red bridge dedicada.

Diseño

- Diseño del diagrama de base de datos con 13 tablas: `empresas`, `dispositivos`, `usuarios_web`, `usuario_empresa`, `personas`, `turnos`, `asignaciones`, `asistencias`, `sincronizacion_log`, `integraciones_erp`, `consentimientos`, `logs_biometricos`, `eliminaciones_biometricas`.
- Diseño de la API REST con 9 blueprints de Flask: `auth`, `personas`, `turnos`, `asignaciones`, `asistencias`, `facial`, `dispositivos`, `logs`, `erp`.
- Diseño del flujo MQTT: tópico único `esp32/imagen/registrar` con payload JSON (`persona_id` + imagen Base64), respuesta en `esp32/respuesta/facial`, heartbeats en `esp32/heartbeat/<MAC>`, LWT en `esp32/lwt/<MAC>`.
- Diseño de endpoints de sincronización y comunicación HTTP con header `X-Device-MAC`.

Implementación

- Implementación de la base de datos con `init_db()` idempotente (`CREATE IF NOT EXISTS` + `ALTER ADD COLUMN`), índices y datos semilla.
- Implementación del cliente MQTT en ESP32-CAM (soporte wss:// con TLS) y en backend Python (`paho-mqtt`) con suscripciones a tópicos de registro facial, heartbeat y LWT.
- Implementación de envío de imagen facial en un único mensaje JSON (sin fragmentación) por MQTT con QoS 1.
- Implementación de `HTTPClient` en ESP32 para comunicación con la API REST (`fetch` personas/turnos/asignaciones, `post` asistencias, `sync` lotes).
- Despliegue Docker Compose con Mosquitto (puertos 1883 y 9001), volúmenes persistentes y red `teleasist_network`.

Pruebas

- Prueba de conexión Wi-Fi con reconexión automática y modo AP como fallback.
- Prueba de envío MQTT de imagen y recepción de respuesta de confirmación.
- Prueba de heartbeat (30s) y LWT (marca como inactivo en <10s).
- Prueba de recepción HTTP de datos en backend para las 4 entidades.
- Prueba de sincronización por lotes de asistencias offline (`POST /api/asistencias/sync`).

- Prueba de inicialización de base de datos desde cero.
- Prueba de latencia de transmisión MQTT extremo a extremo.

3.2.4 Iteración 4: Reconocimiento facial, anti-spoofing y cifrado biométrico

Análisis

- Evaluación de librerías de reconocimiento facial: selección de DeepFace por su soporte multi-modelo y anti-spoofing integrado.
- Selección del modelo Facenet (embeddings de 128 dimensiones, umbral de distancia euclidiana 10.0) y detector RetinaFace (robusto a variaciones de pose e iluminación).
- Identificación de requisitos de seguridad: cifrado de embeddings con Fernet (AES-128 CBC + HMAC-SHA256), consentimiento biométrico obligatorio, detección de registros duplicados.

Diseño

- Diseño del flujo de registro facial: recepción de imagen → verificación de consentimiento → extracción de embedding (sin anti-spoofing) → comparación contra todos los embeddings existentes (umbral 10.0) → cifrado Fernet → persistencia.
- Diseño del flujo de identificación 1:N: recepción de JPEG crudo → guardado de auditoría → extracción de embedding (con anti-spoofing) → comparación contra toda la base → retorno del mejor match.
- Diseño del flujo de verificación 1:1: descifrado de embedding almacenado → comparación con embedding de imagen capturada (anti-spoofing activado) → cálculo de confianza.
- Diseño del flujo de eliminación segura de datos biométricos con conservación de asistencias históricas.

Implementación

- Implementación de 3 endpoints faciales REST (**registrar**, **identificar**, **verificar**) con logging biométrico en cada operación.
- Implementación de procesamiento facial vía MQTT en hilo del backend con verificación de consentimiento y respuesta al ESP32-CAM.

- Implementación de cifrado/descifrado de embeddings con Fernet usando clave derivada de SHA-256 sobre variable de entorno.
- Implementación de detección de identidades duplicadas (HTTP 409) y de endpoint de actualización de rostro.
- Implementación de script CLI `deteccion.py` para simulación de asistencia facial desde fotos almacenadas.

Pruebas

- Prueba de registro facial exitoso con consentimiento y sin duplicados.
- Prueba de detección de rostro duplicado con distancia < 10.0 .
- Prueba de rechazo por falta de consentimiento biométrico (HTTP 403).
- Prueba de identificación 1:N (match correcto entre 3 personas) y de rostro desconocido (HTTP 404).
- Prueba de anti-spoofing con fotografía impresa y pantalla de teléfono.
- Prueba de resistencia a 3 condiciones de iluminación distintas.
- Prueba de cifrado/descifrado de embeddings (ida y vuelta sin pérdida).
- Prueba de eliminación segura de datos biométricos con trazabilidad.

3.2.5 Iteración 5: Módulo antifraude y validación previa a la captura

Análisis

- Identificación de escenarios de suplantación: fotografía impresa, pantalla de dispositivo, ausencia de presencia real.
- **Cambio tecnológico justificado:** El plan original contemplaba un sensor IR dedicado. Sin embargo, la extrema limitación de pines GPIO en el ESP32-CAM —tras la asignación del bus de cámara y el sensor de huellas— forzó la adopción de una estrategia alternativa. Se determinó que el detector anti-spoofing de DeepFace (basado en redes convolucionales) ofrece una precisión comparable o superior a la de sensores IR de bajo costo, y que el sensor PIR (ya presente para detección de presencia) podía asumir la función de “portero” físico del sistema. Esta decisión permitió liberar el GPIO que habría ocupado el IR y fortalecer la capa de seguridad mediante software.

- Enfoque adoptado: estrategia antifraude de 3 capas combinando PIR (detección física de calor corporal), flash PWM controlado (respuesta diferencial del rostro real a fuente de luz puntual) y anti-spoofing por software (DeepFace).

Diseño

- Diseño del PIR como portero: reposo absoluto hasta detección de movimiento → ventana de 15 segundos de actividad → retorno a reposo si no hay más detección.
- Diseño de 3 mecanismos de control: cooldown entre marcaciones (8 s), debounce de huella (4 s) y bloqueo por uso del menú web (30 s).
- Diseño de firma de movimiento: muestreo de 128 bytes del buffer JPEG para detectar cambios entre capturas consecutivas (umbral 1800).
- Diseño de la secuencia de captura con flash PWM: encendido al 50% duty durante 150 ms → captura → apagado inmediato.
- Diseño de señalización visual mediante flash: 2 destellos para éxito, 1 destello largo para error.

Implementación

- Integración del sensor PIR al GPIO12 del ESP32-CAM con pull-down y calibración de 3 segundos en el setup.
- Programación de la máquina de estados de presencia con timer de 15 segundos y delay de estabilización de 800 ms tras detección.
- Programación del control de cooldown, debounce de huella (compara fingerID y timestamp de última lectura) y bloqueo por menú.
- Implementación de la firma de movimiento y la secuencia de flash PWM con apagado post-captura.
- Programación de las funciones `flashExito()` y `flashError()` para retroalimentación sin pantalla.
- Configuración de anti-spoofing en backend para endpoints de identificación y verificación.

Pruebas

- Prueba de detección PIR a 50 cm y retorno a reposo tras 15 s sin presencia.
- Prueba de falsos positivos del PIR en habitación vacía durante 10 minutos.
- Prueba de cooldown entre marcaciones (8 s) y debounce de huella (4 s).
- Prueba de bloqueo por menú durante operaciones administrativas.
- Prueba de anti-spoofing con fotografía impresa y pantalla de teléfono.
- Prueba de flash PWM: duración, duty cycle y calidad de imagen resultante.
- Prueba de señalización por flash (éxito 2 destellos, error 1 destello).

3.2.6 Iteración 6: Autenticación, multi-tenant y enrolamiento de dispositivos

Análisis

- Definición del modelo multi-tenant: múltiples empresas operando sobre una misma instancia del backend con datos aislados mediante columna `empresa_id`.
- Definición de 3 roles jerárquicos: admin (global), empleador (su empresa), trabajador (sus propios datos).
- Requisito de autenticación JWT (HS256, 24h de expiración) para proteger los endpoints del panel web.
- Requisito de enrolamiento seguro de dispositivos ESP32-CAM mediante PIN de 8 caracteres de un solo uso.
- Requisito de monitorización de dispositivos vía heartbeat MQTT (30s) y watchdog (90s timeout).

Diseño

- Diseño del esquema multi-tenant: tabla `empresas` como raíz, `usuario_empresa` como relación muchos-a-muchos con rol.
- Diseño del flujo de autenticación: login → bcrypt → selección de empresa → JWT con `user_id`, `empresa_id`, rol, `persona_id`.
- Diseño de 3 decoradores de protección: `@token_required`, `@token_opcional` (infiere empresa desde X-Device-MAC), `@requiere_rol`.

- Diseño del flujo de enrolamiento: generación de PIN en panel web → ingreso en ESP32-CAM → POST con PIN+MAC+IP → activación.

Implementación

- Implementación del módulo `auth.py` con login, register, me, change-password, CRUD de empresas y usuarios.
- Implementación de validación de contraseñas con `bcrypt` y generación de tokens JWT con `PyJWT`.
- Implementación de control de acceso multi-tenant en todos los blueprints del backend.
- Implementación de enrolamiento de dispositivos con PIN (8 caracteres alfanuméricos, `secrets.choice`) y endpoint `/api/dispositivos/enrolar`.
- Implementación de heartbeat MQTT + LWT + watchdog en `mqtt_handler.py` (hilo daemon cada 60s).
- Implementación de endpoint `/api/dispositivos/verificar` para test de conectividad con el ESP32-CAM.

Pruebas

- Prueba de login exitoso y rechazo de credenciales inválidas.
- Prueba de login con múltiples empresas y selección.
- Prueba de acceso denegado por rol insuficiente (HTTP 403).
- Prueba de aislamiento multi-tenant entre 2 empresas.
- Prueba de generación de PIN y enrolamiento exitoso.
- Prueba de heartbeat (marca activo) y LWT (marca inactivo).
- Prueba de watchdog (inactivo tras 90s sin heartbeat).

3.2.7 Iteración 7: Panel web para la gestión del dispositivo e integración con ERP

Análisis

- Necesidad de un panel web que centralice, en primer lugar, la gestión del dispositivo IoT (enrolamiento, estado de conexión, generación de PIN, verificación

de conectividad, logs de sincronización) y, en segundo lugar, la administración de personas, turnos, asignaciones, ERP, usuarios y empresas.

- Público objetivo: administradores y empleadores (gestión de personas, turnos, dispositivos, ERP); trabajadores (consulta de asistencias propias).
- Definición de modelo de integración ERP vía webhooks salientes: cada asistencia se notifica automáticamente a los ERP configurados en un hilo separado.
- Requisito de field mapping configurable para adaptar el formato JSON de salida al esquema de cada ERP.
- Requisito de test de conectividad manual antes de activar el envío automático.

Diseño

- Diseño del panel web backend con 9 blueprints REST sirviendo datos a un frontend independiente vía CORS.
- Diseño de la tabla `integraciones_erp` con campos: nombre, tipo, `webhook_url`, `headers`, `field_map`, `envio_auto`, activo.
- Diseño del flujo de notificación ERP: `create_asistencia` → hilo daemon → `enviar_asistencia_a_erps()` → field mapping → POST al webhook → registro de estado.
- Diseño del endpoint `GET /api/dispositivos/erp-config` para que el ESP32-CAM descargue su configuración ERP local.

Implementación

- Implementación del módulo `erp.py` con CRUD de integraciones, envío automático asíncrono, test de webhook y envío manual por lotes.
- Implementación de `_transformar_datos()` para field mapping y `_enviar_a_webhook()` con timeout de 10s.
- Implementación de registro de estado de envío (`ultimo_envio`, `ultimo_estado`) en la tabla de integraciones.
- Implementación de sincronización de ERP config hacia el ESP32-CAM (cada hora).
- Configuración de CORS en Flask para permitir frontend externo.

Pruebas

- Prueba de navegación en panel web (endpoints REST respondiendo correctamente con y sin autenticación).
- Prueba de creación de integración ERP y test de webhook con servidor de prueba (webhook.site).
- Prueba de field mapping con campos renombrados.
- Prueba de envío asíncrono (respuesta HTTP al cliente < 200ms independiente de latencia del ERP).
- Prueba de tolerancia a fallos del ERP (HTTP 500 no interrumpe el sistema).
- Prueba de sincronización de ERP config hacia el ESP32-CAM.
- Prueba de envío manual por lote de asistencias históricas.

3.2.8 Iteración 8: Sincronización diferida, logs y cierre del proyecto

Análisis

- Definición de métricas de sincronización: cero pérdida de datos offline, idempotencia ante reenvíos, resolución de conflictos de IDs locales vs. remotos.
- Requisito de auditoría completa: logs de sincronización (`sincronizacion_log`), logs biométricos (`logs_biometricos`) y registro de eliminaciones (`eliminaciones_biometricas`).
- Requisito de watchdog de dispositivos para detectar desconexiones incluso sin LWT.
- Identificación de herramientas de diagnóstico: script `deteccion.py` para simulaciones sin hardware.

Diseño

- Diseño del proceso de sincronización en 2 momentos: al iniciar (`sincronizarPendientes`) y cada 5 minutos en el loop.
- Diseño del orden de sincronización: turnos → asignaciones → asistencias (dependencia de `backend_id`).
- Diseño de deduplicación en backend: verificación de existencia previa en ventana de 60s antes de insertar.

- Diseño de las 3 tablas de auditoría con índices para consultas por persona, dispositivo y fecha.
- Diseño del watchdog: barrido inicial (todos inactivos) + verificación cada 60s con timeout de 90s.

Implementación

- Implementación de `sincronizarTurnosPendientes()`, `sincronizarAsignacionesPendientes` y `sincronizarAsistencias()` en ESP32-CAM.
- Implementación de sincronización bidireccional de personas: descarga desde backend si no hay pendientes locales.
- Implementación de `POST /api/asistencias/sync` con deduplicación por `persona_id` + tipo + ventana temporal.
- Implementación de `device_watchdog()` en hilo daemon con barrido inicial y verificación periódica.
- Implementación de endpoints de consulta y limpieza de logs (`GET/DELETE /api/logs`).
- Implementación de script `deteccion.py` con menú interactivo de simulación facial.
- Integración final de todos los módulos y verificación de flujo punta a punta.

Pruebas

- Prueba de sincronización de 5 asistencias offline → verificación en BD → idempotencia en segundo envío.
- Prueba de sincronización de persona, turno y asignación creados localmente (resolución de IDs locales).
- Prueba de persistencia tras corte de energía durante sincronización.
- Prueba de logs de sincronización (3 ciclos con conteos correctos).
- Prueba de logs biométricos (registro, identificación exitosa, fallida, verificación).
- Prueba del watchdog: barrido inicial → heartbeat → inactividad → marcado inactivo.
- Prueba de simulación facial con `deteccion.py` (match y no-match).

- Prueba de flujo completo punta a punta: registro persona + huella + rostro → sincronización → marcación offline → reconexión → verificación en panel web → envío a ERP de prueba.

3.3 Metodología de evaluación

En esta sección se explica de qué manera se aplicaron, dentro del desarrollo del prototipo, las metodologías de evaluación descritas en el apartado 2.4.2 del marco teórico. Cada técnica de evaluación fue seleccionada en función de los objetivos del proyecto y del enfoque metodológico utilizado prototipado evolutivo con incrementos funcionales, lo que permitió validar el funcionamiento conjunto del hardware, software, backend y mecanismos de autenticación biométrica en entornos reales o simulados. Las evaluaciones se aplicaron de manera iterativa al finalizar cada incremento del prototipo, garantizando que las funcionalidades incorporadas funcionaran correctamente antes de avanzar al siguiente nivel de complejidad.

3.3.1 Pruebas de integración

Las pruebas de integración, como por su nombre dice son pruebas que permiten integrar componentes de un sistema verificando que su interacción sea correcta cuando se implementa un nuevo módulo o dispositivo como es en el contexto de este proyecto. Dado que el proyecto está compuesto por múltiples módulos independientes, lector de huellas, cámara del ESP32-CAM, el almacenamiento local, backend, reconocimiento facial, sistema antifraude (PIR + anti-spoofing), panel web y módulo ERP, es necesario verificar su interoperabilidad a medida que se ensamblan.

Con el objetivo de asegurar que todos los componentes mencionados anteriormente interactúen correctamente entre sí se realizaron pruebas de integración de manera ascendente. El procedimiento que irá realizando el siguiente procedimiento.

1. integración del hardware: Se evaluará la comunicación entre el ESP32-CAM y el módulo de huella digital, Además de comprobar la lectura y envío de datos de sensores IR.
2. integración Almacenamiento: Se realizarán pruebas entorno al almacenamiento de los identificadores de huella, imágenes capturadas además de los registros de asistencia, donde se verificara la consistencia del almacenamiento.
3. integración con el backend: Se enviarán distintos tipos de datos al backend, imágenes en base64, registros de logs, entre otros para verificar la comunicación entre el ESP32 y el backend.

4. integración Backend con la base de datos: Se realizarán inserciones y búsqueda de elementos para verificar la conectividad constante con la base de datos.
5. integración backend con interfaz web: Se comprobará si se rescatan los registros enviados por el prototipo y actualización en tiempo real con los datos.

Estas pruebas permiten evaluar el sistema propuesto de manera sistemática, de manera que cada elemento más pequeño se va integrando a medida que se avanza en el proyecto.

3.3.2 Pruebas de usabilidad

Las pruebas de usabilidad SUS como se menciona anteriormente en el punto 2.4.2 permite evaluar la facilidad de uso, la eficacia del sistema o la claridad de la interfaz. Dado el contexto del proyecto de un prototipo de control de asistencia sus usuarios esperados serían trabajadores y desarrolladores, es por esto que la prueba de usabilidad y facilidad con la que interactúan usuarios no técnicos como trabajadores, empleadores, y junto a esto la facilidad de integración del sistema destinada para programadores o personal técnico que deberá integrar el prototipo con el ERP. Considerando lo anterior el procedimiento será de realizar una encuesta con 10 preguntas, donde se responderán en base a la escala Likert. A continuación se presentarán las preguntas tanto para usuarios no técnicos y para desarrolladores.

Preguntas para usuarios no técnicos

- ¿Qué tan fácil fue encontrar la opción para configurar la red WiFi dentro de la interfaz embebida?
- ¿Lograste modificar los parámetros del dispositivo (como el endpoint del backend) sin necesitar asistencia?
- ¿Te resultó clara la estructura del panel web al momento de buscar registros de asistencia?
- ¿Pudiste identificar rápidamente las imágenes asociadas a cada marcación en el panel web?
- ¿Encontraste intuitivo el uso de los filtros por fecha, usuario o tipo de autenticación?
- ¿La interfaz te entregaba suficiente información para entender si el dispositivo estaba funcionando online u offline?

- ¿En algún momento sentiste confusión respecto a qué botón o sección debías utilizar para continuar una tarea?
- ¿Te pareció clara y legible la información mostrada en la tabla de registros?
- ¿Consideras que podrías aprender a usar la interfaz completa (embebida + panel web) sin instrucción adicional?
- ¿Experimentaste dificultades al intentar visualizar o interpretar algún dato dentro de la interfaz?

Preguntas para usuarios técnicos

- ¿La documentación técnica del backend contenía suficiente información para comenzar a integrar la API sin asistencia externa?
- ¿Pudiste consumir los endpoints del prototipo utilizando herramientas como Postman, cURL o aplicaciones propias sin complicaciones?
- ¿La estructura JSON entregada por el dispositivo te pareció clara, consistente y fácil de mapear a otros sistemas?
- ¿Lograste identificar rápidamente los campos esenciales del registro (timestamp, usuario, métodos biométricos, estado de sincronización)?
- ¿Te resultó sencillo modificar el endpoint configurado en el dispositivo para redirigir los datos hacia tu propio servidor?
- ¿Los mensajes de error entregados por el backend fueron comprensibles y útiles para depurar problemas de integración?
- ¿Encontraste coherencia entre los distintos endpoints del backend (nombres, métodos HTTP, estructura de respuesta)?
- ¿Fue simple integrar o reutilizar las imágenes enviadas por la ESP32-CAM para procesamiento posteriores?
- ¿Consideras que la API del prototipo podría integrarse fácilmente a un sistema ERP real sin necesidad de modificaciones mayores?
- ¿Tuviste dificultades técnicas significativas durante el proceso de integración que consideres relevantes para mejorar el sistema?

3.3.3 Pruebas de confiabilidad y disponibilidad

Para un prototipo IoT con funcionamiento offline, la confiabilidad del almacenamiento local y la disponibilidad del sistema durante fallas de red son esenciales. Estas pruebas evaluarán que tan robusto puede llegar a ser el prototipo frente a escenarios reales de operaciones y fallos. Para el procedimiento se realizará lo siguiente.

- Escenario de conectividad: Para realizar esta prueba se consideran distintos escenarios que permita evaluar el funcionamiento del prototipo en estos escenarios de los cuales consideramos escenarios de desconexión total, conectividad intermitente, reconexión luego de una desconexión prolongada.
- Métricas a evaluar: Dentro de las métricas que se evaluarán corresponden a los porcentajes de registros almacenados sin errores, cantidad de re intentos automáticos, consistencia del log de eventos, tiempo de recuperación tras re conectar (sincronizar datos).
- Verificación con base de datos: Con la finalidad de comprobar la confiabilidad del sistema se debe comparar el número de registros capturados offline en comparación con los almacenados en la base de datos o que fueron sincronizados anteriormente.

3.4 Resultados Esperados

A continuación en esta sección se presentarán los resultados esperados para los objetivos específicos.

- Objetivo específico 2: Implementar el módulo biométrico de registro de asistencia mediante reconocimiento facial y huella digital.
 - Resultado esperado: Para este objetivo se espera tener un prototipo que permita registrar asistencia de un trabajador, mediante reconocimiento facial o huella digital.
- Objetivo específico 3: Incorporar capacidad de funcionamiento offline con almacenamiento local y sincronización posterior.
 - Resultado esperado: Para este objetivo se espera que el prototipo pueda tener la capacidad de funcionamiento offline almacenando los registros de manera local para su posterior sincronización con el servidor.
- Objetivo específico 6: Integrar y validar el funcionamiento conjunto del hardware y software desarrollado.

- Resultado esperado: Para este objetivo se espera que el hardware construido funcione correctamente en conjunto como así su operabilidad con el backend y software.
- Objetivo específico 7: Evaluar el desempeño, confiabilidad y usabilidad del sistema en escenarios de prueba reales.
 - Resultado esperado: Para este objetivo se espera obtener alta disponibilidad con al menos un 90% de su funcionamiento en los escenarios en donde el dispositivo se encuentre offline, o con conexión intermitente.

3.5 Resumen del capítulo

El capítulo 3 describe cómo se aplica la metodología de prototipado evolutivo incremental para el desarrollo del sistema. Se detallan las iteraciones planificadas, cada una incorporando nuevas funcionalidades: configuración inicial del hardware, almacenamiento local y modo offline, comunicación con backend, reconocimiento facial, módulo antifraude con PIR y anti-spoofing, autenticación JWT y multi-tenant, panel web backend, integración con ERP, sincronización diferida y validación final del prototipo.

Asimismo, se explica la metodología de evaluación, que incluye pruebas de integración, usabilidad mediante el método SUS, y pruebas de confiabilidad y disponibilidad para medir el desempeño del sistema en escenarios reales. El capítulo concluye presentando los resultados esperados asociados a los objetivos específicos del proyecto.

4. Desarrollo

En el siguiente capítulo se enfocará en presentar la implementación realizada en base a la metodología de desarrollo.

4.1 Diseño de software

Esta sección está destinada a presentar las arquitecturas desarrolladas para este proyecto como así diagramas y mockups que permitan dar entender la lógica del proyecto como así futuras vistas.

4.1.1 Arquitectura física

La arquitectura física propuesta se basa en una arquitectura basada en microservicios. En la figura 4.1 se puede presenciar como sería la comunicación entre el dispositivo IoT que emite el evento para realizar el registro, para luego que el servidor con el broker MQTT reciba las imágenes y utilice el servicio de reconocimiento facial, para luego enviarlas al servidor backend en donde se distribuyen para el sistema ERP y para la registrarlo en la base de datos, además de disponer del panel web para la visualización de los datos de manera remota.

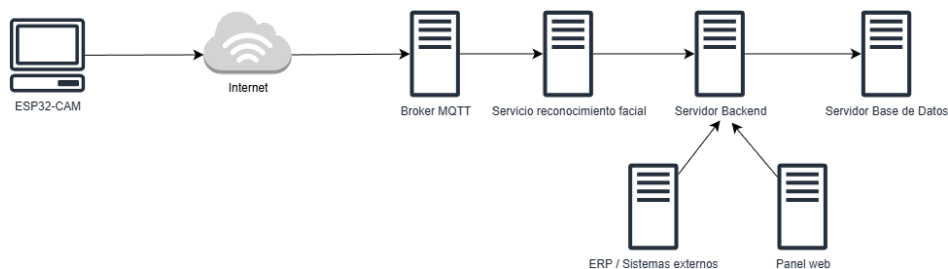


Figura 4.1: Arquitectura física

4.1.2 Arquitectura lógica

La arquitectura lógica del sistema organiza los módulos de software que permiten el funcionamiento del dispositivo IoT y su comunicación con el backend. Tal como se muestra en la Figura 4.2, el ESP32-CAM integra los componentes encargados de la captura de imágenes, la lectura de la huella digital, el almacenamiento local mediante SPIFFS y el servidor web embebido, los cuales interactúan entre sí para registrar usuarios, capturar datos biométricos y almacenar o enviar asistencias.

El backend recibe la información enviada por el dispositivo, procesa las imágenes, administra la base de datos y expone una API que permite la integración con sistemas externos como ERP. Finalmente, la arquitectura lógica contempla la comunicación entre cada módulo mediante protocolos HTTP o MQTT, lo que posibilita el funcionamiento offline, la sincronización posterior y la consulta remota de los datos desde el panel web.

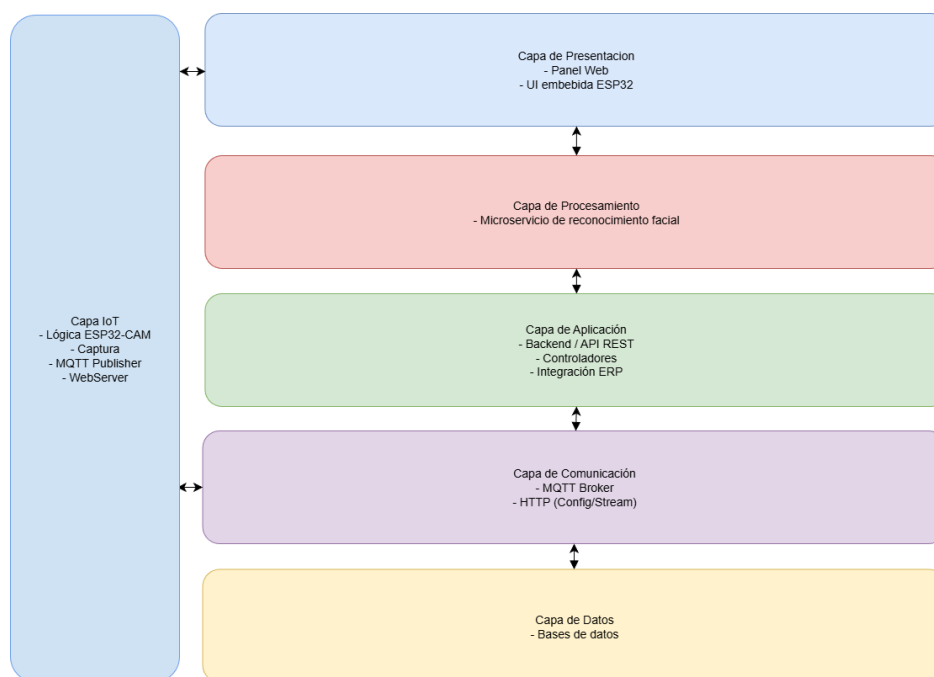


Figura 4.2: Arquitectura lógica

4.2 Implementación

Es este apartado se procederá a explicar como se llevaron a cabo las iteraciones planteadas en el capítulo 3.1.

4.3 Iteración 1: Integración de hardware y servidor embebido

La primera iteración corresponde al diseño conceptual del sistema y a la preparación del entorno de desarrollo, incluyendo la integración inicial del hardware biométrico seleccionado. Esta etapa establece la base funcional sobre la cual se construirán las iteraciones posteriores.

4.3.1 Análisis

La primera iteración se orientó a comprender el alcance inicial del prototipo y a establecer los requisitos mínimos necesarios para validar la viabilidad técnica del sistema. En esta etapa se analizaron los componentes esenciales, las restricciones del hardware y las funcionalidades críticas que debían ser implementadas para permitir la construcción de las siguientes iteraciones.

El análisis se centró en tres ejes principales:

- **Definición de funcionalidades mínimas (MVP):** Se identificaron las capacidades básicas que el dispositivo debía cumplir desde el inicio: capturar imágenes desde la cámara integrada, leer huellas dactilares mediante el sensor AS608, montar un servidor HTTP local con páginas HTML embebidas y operar de forma autónoma mediante Wi-Fi. Estas funciones representan la base operativa sobre la cual se desarrollarán características más avanzadas como turnos, sincronización y gestión de usuarios.
- **Identificación de limitaciones técnicas:** Se revisaron las restricciones tecnológicas del ESP32-CAM, como la limitada memoria RAM disponible (520 KB), el uso compartido del bus de cámara con los pines GPIO, la ausencia de pines GPIO libres —lo que forzó el uso de los pines GPIO14 (RX) y GPIO15 (TX) para la comunicación UART con el lector de huellas— y la necesidad de un manejo eficiente de cadenas HTML grandes en memoria. También se identificó que el flash LED integrado consume corriente significativa (250 mA), requiriendo protección mediante PWM para evitar caídas de tensión durante la captura.
- **Estrategia de operación autónoma:** Dado que uno de los requisitos fundamentales del sistema es la autonomía, se analizó la necesidad de montar un servidor web local. En caso de no contar con Wi-Fi configurada, el dispositivo debe activar automáticamente un modo Access Point (AP) con SSID `ESP32-ASISTENCIA`, permitiendo al administrador conectarse directamente para configurar la red y

registrar usuarios sin dependencia de internet. Esto reduce riesgos y asegura la continuidad operacional del dispositivo.

Como resultado del análisis, se establecieron los criterios de diseño que guiarían el desarrollo de esta iteración: simplicidad, bajo consumo de recursos, modularidad de componentes y autonomía operativa.

4.3.2 Diseño

En esta iteración se elaboraron los primeros elementos arquitectónicos del sistema:

- **Arquitectura física**, representando los componentes utilizados y su interconexión, la cual se puede revisar en la Figura 4.1 del 4.1.1 Arquitectura física.
- **Arquitectura lógica**, definiendo los módulos internos del firmware tales como captura de imágenes, lectura de huella, almacenamiento y conectividad, la cual se puede revisar en la Figura 4.2 del 4.1.2 Arquitectura lógica.
- **Mockups iniciales** de la interfaz embebida del ESP32-CAM, incluyendo:
 - Menú principal.

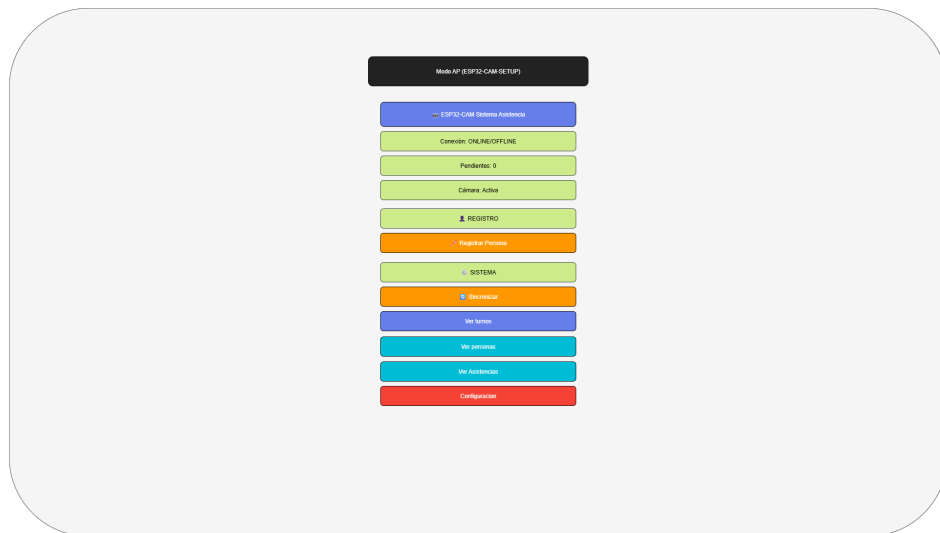


Figura 4.3: Mockup menú principal en el ESP32-CAM

- Panel de configuración de Wi-Fi.

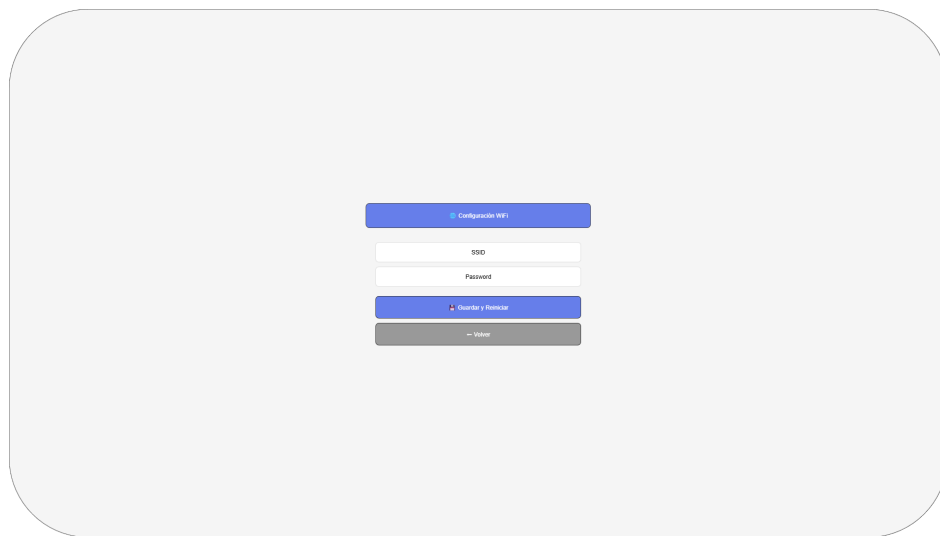


Figura 4.4: Mockup panel de configuración Wi-Fi

- Vista de registro de usuario.



Figura 4.5: Mockup registro usuario en el ESP32-CAM

- Vista de usuarios almacenados localmente.

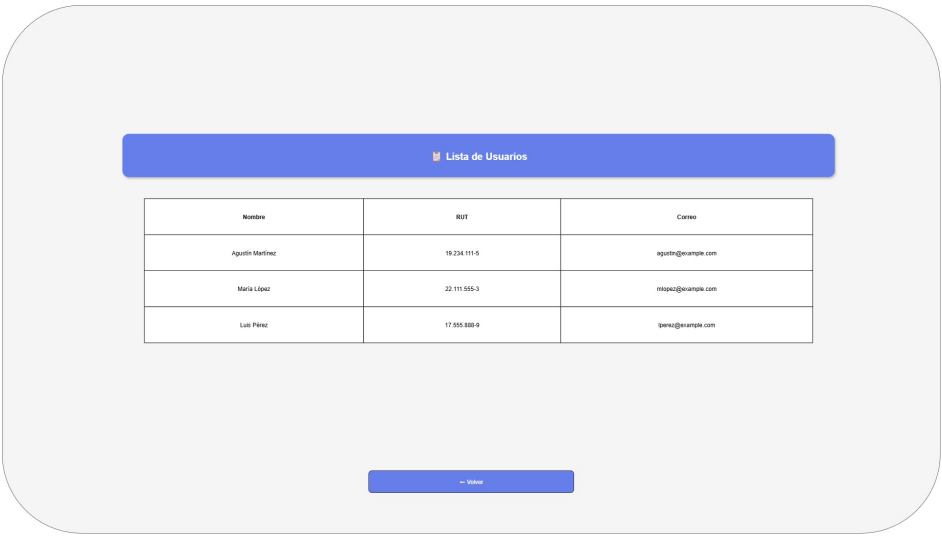


Figura 4.6: Mockup usuarios almacenados en el ESP32-CAM

– Vista de turnos almacenados localmente.

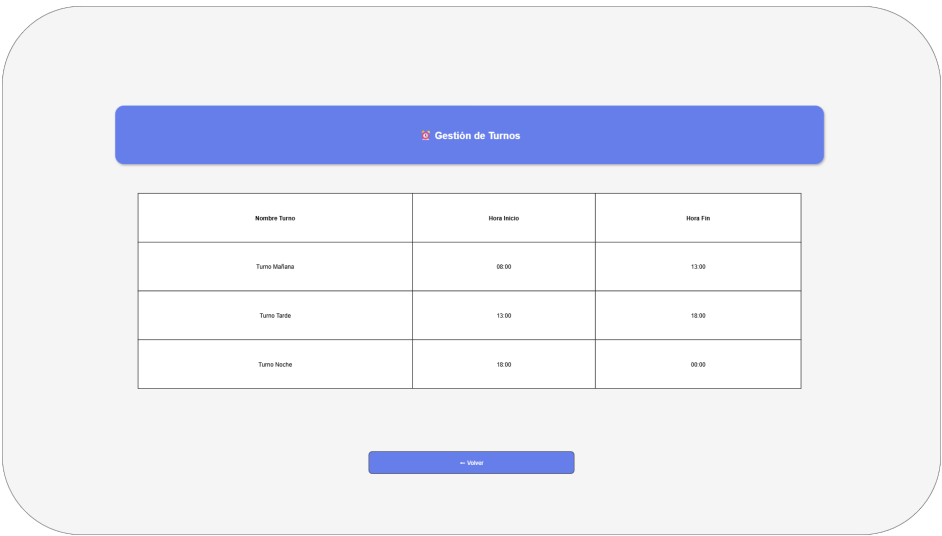


Figura 4.7: Mockup visualización de turnos en el ESP32-CAM

– Vista de crear turnos.

The mockup for creating shifts is displayed on a light gray background. It features a blue header bar labeled "Gestionar turnos". Below this, there is a form with three input fields: "Nombre del turno:" followed by a wide text box, "Hora Inicio:" with a time picker showing "08:00", and "Hora Fin:" with a time picker showing "13:00". A second blue header bar labeled "Turnos asignados" is positioned below the form. Underneath, there is a white box labeled "Lista turnos creados". At the bottom, there are two buttons: a green "Crear Turno" button and a red "Cancelar" button.

Figura 4.8: Mockup creación de turnos en el ESP32-CAM

– Vista de Asignación de turnos.

The mockup for assigning shifts is displayed on a light gray background. It features a blue header bar labeled "Asignar Turno". Below this, there are two selection fields: "Seleccionar Usuario" with a user icon and "Seleccionar Turno" with a calendar icon. A green "Guardar Asignación" button is located below these fields. A second blue header bar labeled "Asignaciones actuales" is positioned below the button. Underneath, there is a white box labeled "Lista de asignaciones actuales". At the bottom, there is a red "Volver" button.

Figura 4.9: Mockup asignación de turnos en el ESP32-CAM

- **Diseño del flujo operacional:** Representa el flujo del proceso de marcación para los casos en se cuenta con conexión a internet y sin internet.

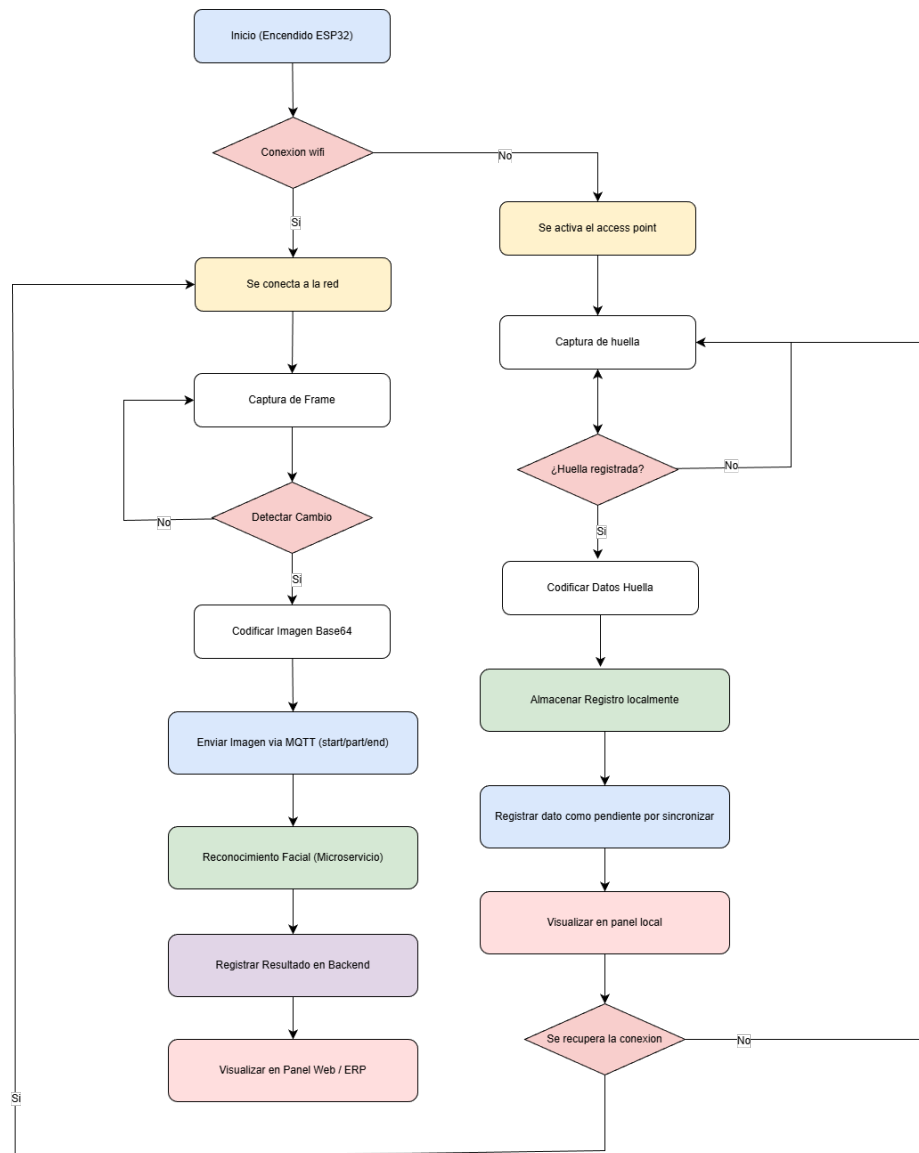


Figura 4.10: Flujo operacional para el registro de asistencia.

Estos elementos permiten visualizar el flujo básico del dispositivo antes de incorporar funcionalidades adicionales como turnos, sincronización o panel web backend, los cuales se desarrollarán en iteraciones posteriores.

4.3.3 Implementación

En esta etapa se integró tanto el hardware principal del prototipo como el desarrollo de las vistas web embebidas que permiten la interacción del usuario con el dispositivo.

Para ello se utilizaron las siguientes librerías:

- `esp_camera`, utilizada para inicializar y configurar la cámara OV2640 integrada en el ESP32-CAM.
- `HardwareSerial` y `Adafruit_Fingerprint`, empleadas para la comunicación UART con el lector biométrico AS608.
- `WebServer`, utilizada para montar un servidor web HTTP en el puerto 80 y atender rutas asociadas a las vistas.
- `LittleFS` y `ArduinoJson`, empleadas para almacenamiento persistente en memoria flash y manipulación estructurada de datos JSON.

Integración del lector de huellas AS608

El lector biométrico AS608 fue conectado directamente al ESP32-CAM mediante la interfaz UART secundaria (`HardwareSerial 2`), utilizando los pines GPIO14 (RX) y GPIO15 (TX). Esta conexión, que emplea pines distintos a los documentados inicialmente, fue forzada por la escasez de GPIO libres tras la asignación del bus de cámara. La velocidad de comunicación se estableció en 57600 baudios, y la librería `Adafruit_Fingerprint` se vincula a dicho puerto serial. La Figura 4.11 presenta la conexión física entre ambos dispositivos.

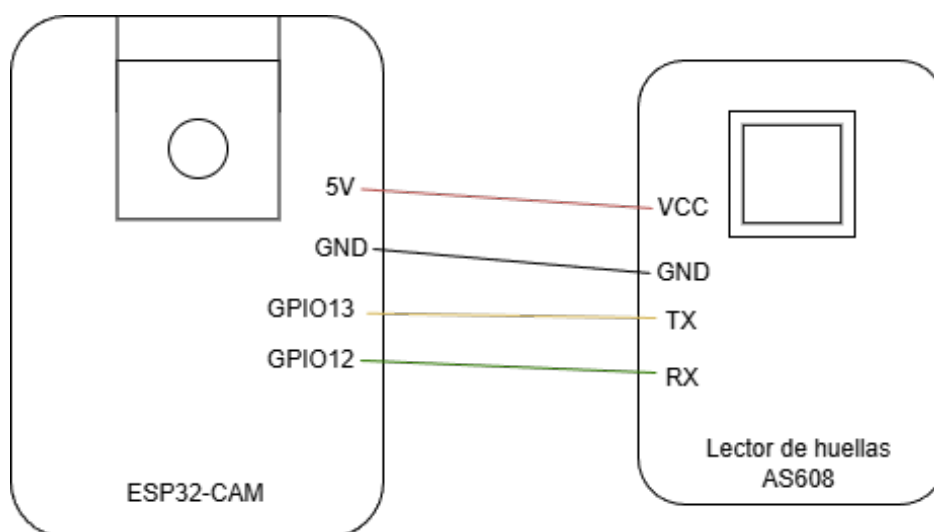


Figura 4.11: Conexiones entre el ESP32-CAM y el lector de huellas AS608.

Una vez establecida la comunicación, el firmware puede ejecutar operaciones como:

- Captura de imagen dactilar.
- Conversión de la imagen en una plantilla biométrica.
- Almacenamiento del template en un slot interno del sensor (1–127).
- Identificación mediante búsqueda de coincidencias en memoria.

El fragmento de código que se ve en la Figura 4.12 es un ejemplo que se construyó para ser utilizado en iteraciones posteriores para el uso del lector de huellas junto al ESP32-CAM.

```
HardwareSerial FingerSerial(2);
Adafruit_Fingerprint finger = Adafruit_Fingerprint(&FingerSerial);

void initFinger() {
    FingerSerial.begin(57600, SERIAL_8N1, 12, 13);
    finger.begin(57600);

    if (finger.verifyPassword()) {
        Serial.println("Sensor AS608 listo.");
    } else {
        Serial.println("Error: no se detecta el AS608");
    }
}
```

Figura 4.12: Fragmento de código para inicializar el lector AS608 utilizando UART.

Configuración de la cámara y el flash PWM

La cámara OV2640 se configuró con resolución VGA (640×480) y compresión JPEG con calidad 8 (escala 0–63, donde valores bajos indican mayor calidad). La frecuencia del reloj externo (XCLK) se fijó en 20 MHz. Para evitar el parpadeo visible y las caídas de tensión provocadas por el flash LED durante la captura, se implementó un control PWM a 5 kHz con resolución de 8 bits, aplicando un ciclo de trabajo del 50% (duty 128 de 255) durante la iluminación. Esto permite encender el flash con brillo suficiente sin sobrecargar la fuente de alimentación del ESP32-CAM.

Calibración del sensor PIR

Se integró un sensor PIR (Passive Infrared) conectado al GPIO12 configurado con resistencia de pull-down interna. El sensor PIR actúa como detector de presencia: cuando una persona se sitúa frente al dispositivo, el PIR detecta el cambio de radiación

infrarroja y activa el modo de captura. El setup incluye una fase de calibración de 3 segundos para estabilizar la lectura del sensor antes de iniciar el bucle principal. La lógica de uso del PIR como control de activación se detalla en la iteración 6.

Montaje del servidor web embebido

Para habilitar la interacción local con el dispositivo, se implementó un servidor web mediante la librería **WebServer** en el puerto 80. Este servidor permite entregar páginas HTML, datos JSON y recursos estáticos. Cuando no hay conectividad a una red externa, el ESP32-CAM activa automáticamente un Access Point con SSID **ESP32-ASISTENCIA** y contraseña **Asistencia2026**, sirviendo las mismas páginas sin depender de internet.

Las rutas principales del sistema incluyen:

- `/`: Menú principal con indicadores de estado.
- `/wifi-setup`: Configuración de red Wi-Fi y parámetros del backend.
- `/register`: Registro de personas con captura biométrica.
- `/asistencias`: Lista de asistencias almacenadas localmente.
- `/gestion`: Gestión de turnos (creación y administración).
- `/personas`: Visualización de personas registradas.
- `/asignaciones`: Vista para realizar asignaciones de turnos.
- `/logs`: Visualización de logs del sistema.
- `/turnos`: Visualización de turnos creados.

Para la implementación del servidor HTTP embebido, se utilizó el módulo **WebServer** del ESP32. Cada operación del sistema (registro de usuarios, captura de huellas, creación de turnos, asignación de turnos, lectura de asistencias, configuración Wi-Fi, entre otras) fue mapeada a un endpoint específico mediante funciones manejadoras (*handler functions*). Este mecanismo permite encapsular la lógica de cada componente del sistema y mantener una estructura modular del firmware.


```

server.on("/", []() { server.send(200, "text/html", htmlPage); });
server.on("/register", []() { server.send(200, "text/html", registerPage); });
server.on("/gestion", []() { server.send(200, "text/html", gestionPage); });
server.on("/personas", []() { server.send(200, "text/html", personasPage); });
server.on("/asistencias", []() { server.send(200, "text/html", asistenciasPage); });
server.on("/turnos", []() { server.send(200, "text/html", turnosPage); });
server.on("/asignaciones-view", []() { server.send(200, "text/html", asignacionesPage); });
server.on("/wifi-setup", []() { server.send(200, "text/html", wifiConfigPage); });
server.on("/wifi-config", handleWiFiConfig);
server.on("/registrar", handleRegisterUser);
server.on("/crear_turno", handleCreateTurn);
server.on("/asignar", handleAssignTurn);
server.on("/marcar", handleMarcarAsistencia);
server.on("/api/personas", handleGetPersonas);
server.on("/api/turnos", handleGetTurnos);
server.on("/api/asignaciones", handleGetAsignaciones);
server.on("/api/asistencias", handleGetAsistencias);
server.on("/limpiar", handleLimpiarDatos);

server.begin();

```

Figura 4.13: Asignación de rutas para el servidor web embebido.

Creación de vistas HTML embebidas

Las vistas fueron desarrolladas directamente en el firmware utilizando literales de cadena crudos (`R"rawliteral(...)"`), conteniendo HTML, CSS y JavaScript autónomo, sin dependencias de CDN externas. Esto garantiza que el ESP32-CAM sirva cada página incluso en modo Access Point sin conexión a internet.

Menú principal

La Figura 4.14 presenta la vista principal accesible en la ruta `/`. Desde esta interfaz el usuario puede visualizar el estado del dispositivo (online/offline, sensor activo, enrolado), registrar personas, ver asistencias, gestionar turnos, ver asignaciones, acceder a la configuración de red, sincronizar con el backend y limpiar datos del dispositivo.

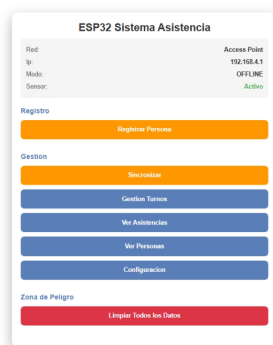


Figura 4.14: Menú principal del sistema web embebido.

Vista de configuración Wi-Fi

Mediante la ruta `/wifi-setup`, el dispositivo ofrece un formulario para actualizar las credenciales de la red local (SSID, contraseña), la URL del backend, la dirección del broker MQTT y el PIN de enrolamiento. Estos valores se persisten en SPIFFS mediante el archivo `wifi.json`. La Figura 4.15 muestra esta interfaz.



Figura 4.15: Interfaz de configuración Wi-Fi.

Vista de registro de personas

La ruta `/register` despliega una vista que combina un formulario para ingresar el nombre del usuario, RUT y correo electrónico, y muestra indicadores visuales del estado del proceso de enrolamiento (huella, rostro). La Figura 4.16 lo muestra.

Registrar Persona

Nombre Completo *

Ej. Juan Perez

RUT *

Ej. 12345678-9

Email *

Ej. juan@email.com

Registrar con Huella

Ver Personas

Volver

Complete los datos y presione Registrar

Vista Previa Camara

Camara no disponible en modo offline.
Funcionalidad reservada para futuras versiones.

Template para integracion futura

Figura 4.16: Vista para el registro de personas en el dispositivo.

Vista de asistencias

Para visualizar registros de asistencia tanto locales como sincronizados, se implementó la vista en la ruta `/asistencias`. Esta consulta archivos JSON almacenados en LittleFS. La Figura 4.17 lo ejemplifica.



Figura 4.17: Vista de asistencias locales almacenadas en el dispositivo.

Vista de Personas registradas

Para visualizar las personas registradas, se implementó la vista en la ruta `/personas`. Esta consulta archivos JSON almacenados en LittleFS y permite editar nombre/correo, actualizar huella o rostro, y eliminar personas. La Figura 4.18 lo ejemplifica.



Figura 4.18: Vista de personas registradas en el dispositivo.

Vista de Crear Turnos

Para la creación de turnos y visualización de los existentes, se implementó la vista en la ruta `/gestion`, complementada con `/turnos`. Estas consultan archivos JSON almacenados en LittleFS. La Figura 4.19 lo ejemplifica.

Gestion de Turnos

Crear Nuevo Turno

Nombre del turno (ej. Turno Mañana)

08:00

16:00

L,M,X,J,V

Crear Turno

Turnos Existentes

Turnos de ejemplo:

- Turno Mañana
08:00 - 16:00 L,M,X,J,V ID: 1
- Turno Tarde
14:00 - 22:00 L,M,X,J,V ID: 2
- Turno Noche
20:00 - 04:00 L,M,X,J,V ID: 3
- Turno Completo
08:00 - 16:00 L,M,X,J,V ID: 4

Volver

Figura 4.19: Vista de gestión de turnos en el dispositivo.

Vista de asignación de turnos

Para visualizar las asignaciones de turnos y realizar la acción de asignar un turno a una persona, se implementó la vista en la ruta `/asignaciones`. La Figura 4.20 lo ejemplifica.



Figura 4.20: Vista de gestión de asignaciones en el dispositivo.

4.3.4 Pruebas

Para validar el funcionamiento del prototipo y comprobar la correcta integración entre los distintos módulos del sistema, se diseñaron pruebas de integración enfocadas exclusivamente en los componentes implementados durante esta iteración:

- **Prueba de captura de imágenes:** Verificar que la cámara OV2640 capture imágenes JPEG de forma continua y estable, sin corrupción de frames ni errores de inicialización del sensor. Se espera que la función `esp_camera_fb_get()` retorne buffers consistentes tras cada llamada.
- **Prueba de comunicación UART con AS608:** Verificar que el lector biométrico responda correctamente al comando de verificación de contraseña (`finger.verifyPassword()`) y que la comunicación serie a 57600 baudios se establezca sin pérdida de paquetes. Se espera confirmación de conexión en menos de 1 segundo tras el inicio.
- **Prueba de lectura biométrica:** Verificar que el AS608 pueda capturar una imagen dactilar (`finger.getImage()`), convertirla a plantilla (`finger.image2Tz()`) y almacenarla en un slot del sensor (`finger.storeModel()`). Se espera que el proceso completo tome menos de 2 segundos.
- **Prueba de acceso vía AP:** Verificar que al iniciar sin credenciales Wi-Fi configuradas, el ESP32-CAM active el Access Point **ESP32-ASISTENCIA** y que las páginas HTML sean accesibles desde un navegador conectado a la IP 192.168.4.1. Se espera que todas las rutas definidas respondan con código HTTP 200.

- **Prueba de carga de vistas vía Wi-Fi externa:** Verificar que al conectar el dispositivo a una red Wi-Fi configurada, las mismas vistas se carguen correctamente a través de la IP asignada por el router, manteniendo estilos, scripts y funcionalidad.

4.4 Iteración 2: Almacenamiento local y modo offline

La segunda iteración se centra en habilitar el funcionamiento autónomo del dispositivo sin conexión a internet, integrando un sistema de almacenamiento persistente basado en LittleFS (sucesor moderno de SPIFFS para ESP32) y los mecanismos de captura biométrica offline. Esta etapa materializa lo definido en el apartado 3.1.2 del capítulo anterior.

4.4.1 Análisis

Para el correcto funcionamiento offline del prototipo se identificaron los siguientes requerimientos:

- Almacenar localmente los datos de usuarios, turnos, asignaciones y asistencias en formato JSON.
- Permitir el registro de marcaciones aun cuando no exista conectividad externa, tanto por huella (offline total) como con validación de turnos locales.
- Garantizar persistencia de la información tras reinicios del ESP32-CAM o cortes de energía.
- Habilitar la gestión completa del sistema mediante una interfaz web embebida accesible vía Access Point.
- Mantener compatibilidad con la futura sincronización automática en modo online, marcando cada registro con un campo **sincronizado** (booleano) para su posterior envío al backend.

Para cumplir estos requisitos se seleccionó el sistema de archivos **LittleFS** y el formato **JSON** debido a su bajo peso, facilidad de manipulación mediante la librería **ArduinoJson** y compatibilidad con las capacidades del ESP32-CAM (4 MB de flash, de los cuales aproximadamente 1.5 MB están disponibles para LittleFS). Adicionalmente, se definió un archivo de configuración, **wifi.json**, destinado a almacenar credenciales Wi-Fi, URL del backend, dirección del broker MQTT y PIN de enrolamiento.

4.4.2 Diseño

Considerando la arquitectura establecida, se diseñaron las siguientes estructuras de almacenamiento en formato JSON:

- **personas.json**: Arreglo de objetos con campos `id`, `nombre`, `rut`, `email`, `huella_id`, `fecha_registro`, `sincronizado`.
- **turnos.json**: Arreglo de objetos con campos `id`, `backend_id`, `nombre`, `inicio`, `fin`, `dias`, `sincronizado`.
- **asignaciones.json**: Arreglo de objetos con campos `persona_id`, `turno_id`, `backend_id`, `fecha_asignacion`, `sincronizado`.
- **asistencias.json**: Arreglo de objetos con campos `persona_id`, `nombre`, `tipo` (entrada/salida), `metodo` (huella_offline, facial, etc.), `timestamp`, `sincronizado`.
- **wifi.json**: Objeto con campos `ssid`, `pass`, `backend`, `mqtt`, `pin` (para enrolamiento del dispositivo).
- **erp-config.json**: Arreglo con configuraciones de integraciones ERP, `sincronizado` desde el backend cuando hay conectividad.

El campo `sincronizado` (booleano) en cada entidad actúa como bandera para el proceso de sincronización diferida: cuando el dispositivo recupera conectividad, itera sobre los registros con `sincronizado = false` y los envía al backend. Una vez confirmada la recepción, se marca `sincronizado = true`.

4.4.3 Implementación

En esta iteración se integraron las librerías `LittleFS`, `ArduinoJson`, `HardwareSerial` y `Adafruit_Fingerprint`. Las dos primeras permiten gestionar archivos JSON alojados en la memoria flash del ESP32-CAM, mientras que las dos últimas habilitan la comunicación UART con el lector biométrico AS608 para el registro y verificación de huellas digitales.

Inicialización del sistema de archivos y persistencia

El primer componente de la implementación consiste en montar `LittleFS` mediante `LittleFS.begin(true)` y verificar la existencia de los archivos necesarios. En caso de no existir, el sistema los crea con una estructura inicial vacía (arreglo `[]` para las colecciones, objeto con valores por defecto para `wifi.json`), asegurando que el dispositivo pueda operar incluso tras un primer arranque sin configuración previa.

Se implementaron dos funciones genéricas para la manipulación de archivos JSON:

- `loadArray(const char* path, DynamicJsonDocument& doc)`: Abre un archivo, deserializa su contenido como `JsonArray` y lo retorna. Si el archivo no existe o está corrupto, retorna un arreglo vacío.
- `saveArray(const char* path, DynamicJsonDocument& doc)`: Serializa el documento JSON y lo escribe en el archivo correspondiente, sobrescribiendo el contenido anterior.

Estas funciones encapsulan toda la lógica de persistencia del sistema y son reutilizadas por todos los módulos que necesiten leer o escribir datos.

```
JsonArray loadArray(const char* path, DynamicJsonDocument &doc) {
    File file = SPIFFS.open(path, "r");
    String content = file.readString();
    file.close();
    deserializeJson(doc, content);
    return doc.as<JsonArray>();
}

void saveArray(const char* path, DynamicJsonDocument &doc) {
    File file = SPIFFS.open(path, "w");
    serializeJson(doc, file);
    file.close();
}
```

Figura 4.21: Funciones para lectura y escritura de archivos JSON en LittleFS.

Registro de usuarios con huella digital

El registro de usuarios se implementó mediante un flujo que combina la captura biométrica gestionada por el lector AS608 con el almacenamiento en `personas.json`. El lector administra internamente modelos de huellas mediante *slots* numerados del 1 al 127. Por ello, antes del enrolamiento es necesario identificar un slot disponible dentro del sensor. La función `encontrarSlotLibre()` recorre el arreglo de personas, determina el ID de huella más alto registrado y retorna el siguiente disponible.

Una vez encontrado un slot, se ejecuta el proceso de enrolamiento en dos fases:

1. **Primera captura** (`ESTADO_ESPERANDO_HUELLA_REGISTRO`): El sistema espera que el usuario coloque el dedo. Al detectar una imagen válida, la convierte a plantilla en el buffer 1 y transiciona a `ESTADO_ESPERANDO_SOLTAR_DEDO`.
2. **Espera y segunda captura** (`ESTADO_REGISTRO_SEGUNDA_HUELLA`): Una vez que el sensor detecta que el dedo fue retirado (`FINGERPRINT_NOFINGER`), el sistema solicita una segunda captura del mismo dedo. Al obtenerla, convierte a plantilla

en el buffer 2, crea el modelo combinado (`finger.createModel()`) y lo almacena en el slot correspondiente (`finger.storeModel()`).

Luego del enrolamiento exitoso, se almacena un nuevo objeto dentro de `personas.json` con un identificador único (generado localmente con prefijo `local-` o provisto por el backend si hay conectividad), el nombre, RUT, correo electrónico y el slot asignado.

Registro de asistencia en modo offline

El registro de asistencia se realiza completamente en el dispositivo, sin necesidad de conexión externa. El flujo implementado consiste en:

1. Capturar una huella mediante el lector AS608 (`finger.getImage()`).
2. Convertir la imagen a plantilla (`finger.image2Tz()`) y ejecutar una búsqueda biométrica (`finger.fingerSearch()`) que retorna el `fingerID` del slot coincidente.
3. Identificar al usuario asociado mediante el campo `huella_id` en `personas.json` con la función `buscarPersonaPorHuella()`.
4. Verificar que el usuario cuente con un turno asignado consultando `asignaciones.json` mediante `turnoActivo()`. Si no tiene turno, el sistema rechaza la marcación.
5. Determinar el tipo de evento (entrada/salida) alternando automáticamente respecto al último registro de la persona en `asistencias.json`.
6. Generar un nuevo registro con `sincronizado = false` (salvo que se logre enviar al backend en ese momento).

El manejador `handleMarcarAsistencia()` expone esta funcionalidad a través de la interfaz web, permitiendo también marcaciones manuales. El escaneo automático continuo se controla mediante intervalos configurables: 2 segundos entre verificaciones de huella y 6 segundos entre verificaciones faciales.

Gestión de turnos y asignaciones

Además del registro de asistencias, el sistema permite crear turnos (`handleCreateTurn()`) y asignarlos a personas (`handleAssignTurn()`) desde la interfaz web. Cada operación persiste los datos en sus archivos JSON correspondientes y, si hay conectividad, intenta replicar la operación en el backend mediante HTTP POST.

Operación offline del dispositivo

Con el objetivo de asegurar autonomía total, el ESP32-CAM fue configurado para operar en dos modos:

- **Modo Wi-Fi STA:** Si hay credenciales configuradas en `wifi.json`, el dispositivo intenta conectarse a la red externa. Si la conexión falla reiteradamente (5 intentos), activa el modo AP como fallback sin deshabilitar el intento de reconexión.
- **Modo Access Point:** El punto de acceso utiliza el SSID `ESP32-ASISTENCIA` y contraseña `Asistencia2026`, permitiendo que un administrador se conecte directamente al dispositivo en la IP 192.168.4.1 para realizar todas las operaciones mediante la interfaz web embebida.

4.4.4 Pruebas

Se diseñaron las siguientes pruebas para validar el correcto funcionamiento del almacenamiento local y el modo offline:

- **Prueba de persistencia tras reinicio:** Registrar usuarios, turnos, asignaciones y asistencias, luego reiniciar el ESP32-CAM (por corte de energía o comando software). Verificar que todos los archivos JSON conserven su contenido íntegro y que el sistema pueda leerlos correctamente tras el reinicio.
- **Prueba de lectura y deserialización:** Abrir cada archivo JSON generado y verificar que `ArduinoJson` pueda deserializar su contenido sin errores ni pérdida de campos. Se espera que el tamaño de cada documento no exceda los límites del `DynamicJsonDocument` configurado (2048–8192 bytes según el archivo).
- **Prueba de escritura concurrente:** Realizar múltiples operaciones de escritura (registro de persona + asignación de turno + marcación de asistencia) en rápida sucesión sin reiniciar el sistema. Verificar que no se produzcan corrupciones de archivos ni pérdida de datos.
- **Prueba de marcación offline por huella:** Desconectar el dispositivo de toda red externa, registrar un usuario con huella y turno, y efectuar una marcación. Verificar que el registro aparezca en `asistencias.json` con `sincronizado = false`.
- **Prueba de alternancia entrada/salida:** Realizar dos marcaciones consecutivas para un mismo usuario. Verificar que el sistema registre correctamente “entrada” seguido de “salida”.

- **Prueba de validación de turno offline:** Intentar marcar asistencia para un usuario sin turno asignado. Verificar que el sistema rechace la marcación con el mensaje “Sin turno asignado”.
- **Prueba de funcionamiento del AP:** Desconectar el ESP32-CAM de cualquier red, verificar que el AP ESP32-ASISTENCIA se active automáticamente y que todas las operaciones de gestión (registro, turnos, asignaciones, asistencias) funcionen correctamente desde un navegador conectado a la IP 192.168.4.1.

4.5 Iteración 3: Backend, base de datos y comunicación

El propósito de esta iteración consiste en implementar un backend centralizado que permita procesar la información enviada por el ESP32-CAM, administrar la base de datos, gestionar registros de trabajadores y turnos, y habilitar la comunicación bidireccional mediante MQTT y HTTP. Esta etapa corresponde a la construcción del servidor del sistema, incorporando funcionalidades que serán extendidas en iteraciones posteriores.

4.5.1 Análisis

En esta iteración se realiza el estudio de los mecanismos de comunicación necesarios para establecer el flujo de datos entre el dispositivo ESP32-CAM y el backend del sistema, considerando tanto la operación en línea como escenarios donde la conectividad es limitada o intermitente.

En primer lugar, se evaluaron los protocolos **HTTP** y **MQTT** como alternativas principales para el envío de datos. MQTT fue seleccionado como el canal preferente para la transmisión de imágenes faciales debido a su naturaleza ligera, eficiencia en dispositivos IoT y capacidad de mantener sesiones persistentes con bajo consumo energético, utilizando WebSockets (`wss://`) como transporte cuando el broker lo soporta. HTTP se mantiene como protocolo complementario para la exposición de servicios REST, operaciones CRUD sobre las entidades del sistema (personas, turnos, asignaciones, asistencias) y procesos de sincronización. Esta combinación permite equilibrar confiabilidad, rendimiento y compatibilidad con otras plataformas.

Asimismo, se analizó la **conectividad Wi-Fi** como medio de enlace principal del dispositivo. En escenarios con conexión estable, el ESP32-CAM transmite imágenes al broker MQTT y consultas HTTP al backend; mientras que, en situaciones de baja o nula conectividad, el dispositivo almacena registros localmente (Iteración 2) y los reenvía cuando la red se restablece. Se definieron mecanismos de reconexión progresiva con backoff exponencial (3, 6, 9, 12 y 15 segundos entre intentos), manejo

de desconexiones mediante el evento `ARDUINO_EVENT_WIFI_STA_DISCONNECTED` y un watchdog que intenta restaurar la conectividad periódicamente.

Finalmente, se evaluó la compatibilidad del flujo de datos con la arquitectura general definida en el apartado 4.1.1, determinando que esta iteración corresponde a la habilitación de la capa de comunicación que enlaza el dispositivo con el backend.

4.5.2 Diseño

El diseño de esta iteración se centra en la definición detallada de los mecanismos de comunicación y conectividad, así como en la arquitectura de la base de datos que soportará todas las funcionalidades del sistema.

El backend se organiza de forma modular mediante **blueprints de Flask**, cada uno responsable de un dominio específico del sistema: personas, turnos, asignaciones, asistencias, reconocimiento facial, dispositivos, logs, integraciones ERP y autenticación. Esta separación facilita el mantenimiento y la escalabilidad del código.

La API REST se diseña para soportar las funcionalidades de gestión operativa del sistema. Los endpoints definidos para esta iteración incluyen:

- `GET /api/personas`: Obtiene el listado de personas registradas (con filtro por roles y empresa en iteraciones futuras).
- `POST /api/personas`: Registra una nueva persona (nombre, RUT, email, huella_id).
- `GET /api/turnos`: Devuelve todos los turnos almacenados.
- `POST /api/turnos`: Permite crear un nuevo turno indicando nombre, hora de inicio, hora de término y días.
- `DELETE /api/turnos/<id>`: Elimina un turno específico.
- `GET /api/asignaciones`: Lista todas las asignaciones activas entre personas y turnos.
- `POST /api/asignaciones`: Asigna un turno a una persona.
- `DELETE /api/asignaciones/<id>`: Elimina una asignación.
- `POST /api/asistencias`: Registra una asistencia enviada desde el dispositivo.
- `POST /api/asistencias/sync`: Recibe un lote de asistencias acumuladas en modo offline y las inserta con deduplicación.

Todos estos endpoints operan sobre una base de datos relacional PostgreSQL. El diseño del esquema incluye las siguientes tablas:

- **empresas:** Entidad multi-tenant con nombre, RUT empresa, email de contacto, teléfono y dirección.
- **dispositivos:** Registro de cada ESP32-CAM enrolado, con MAC address, IP local, estado (activo/inactivo), último heartbeat y código de enrolamiento (PIN de 8 caracteres).
- **usuarios_web:** Usuarios del panel web backend con autenticación por email y contraseña (hash bcrypt).
- **usuario_empresa:** Tabla de relación muchos-a-muchos entre usuarios y empresas, con rol (admin, empleador, trabajador).
- **personas:** Trabajadores registrados con nombre, RUT (único), email, huella_id, encoding facial (cifrado) y empresa asociada.
- **turnos:** Jornadas laborales con nombre, hora de inicio, hora de fin, días y empresa asociada.
- **asignaciones:** Relación entre personas y turnos, con vigencia.
- **asistencias:** Registros de marcación con persona, dispositivo, tipo (entrada/salida), método, timestamp, origen y estado de sincronización.
- **sincronizacion_log:** Bitácora de procesos de sincronización con conteo de registros enviados y procesados correctamente.
- **integraciones_erp:** Configuración de webhooks para integración con sistemas ERP externos (nombre, tipo, URL, headers, field mapping, envío automático).
- **consentimientos:** Registro de aceptación de política de privacidad biométrica por persona.
- **logs_biometricos:** Auditoría de operaciones biométricas (registro, verificación, identificación) con resultado, timestamp e IP de origen.
- **eliminaciones_biometricas:** Registro histórico de datos biométricos eliminados (embeddings anteriores, fotos) para cumplimiento normativo y trazabilidad.

El diagrama de la base de datos se presenta en la Figura 4.28.

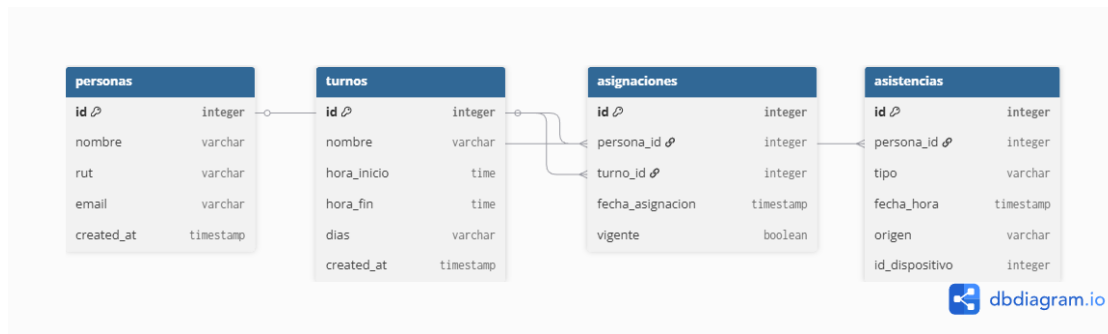


Figura 4.22: Diagrama de la base de datos relacional.

La comunicación MQTT se diseña para transmitir imágenes faciales desde el ESP32-CAM hacia el backend en un único mensaje JSON por el tópico `esp32/imagen/registrar`, conteniendo el `persona_id` y la imagen codificada en Base64. El backend, mediante un cliente MQTT (librería `paho-mqtt` de Python), se suscribe a este tópico, decodifica la imagen y procesa el enrolamiento facial.

Adicionalmente, se definen los tópicos `esp32/heartbeat/<MAC>` para monitorizar la actividad de los dispositivos y `esp32/lwt/<MAC>` para detectar desconexiones mediante el mecanismo de Last Will Testament (LWT). El backend recibe las respuestas del procesamiento facial por el tópico `esp32/respuesta/facial`, que el ESP32-CAM escucha para confirmar el éxito o fallo del registro.

La infraestructura se complementa con **Docker Compose**, que levanta el broker Mosquitto en un contenedor independiente, exponiendo el puerto 1883 para MQTT nativo y el puerto 9001 para WebSockets.

4.5.3 Implementación

La implementación de la Iteración 3 se centró en habilitar la comunicación entre el dispositivo ESP32-CAM y el backend desarrollado en Python con Flask, así como en construir la base de datos PostgreSQL que soporta todo el sistema.

Inicialización de la base de datos

Se implementó el módulo `database.py` con una función `init_db()` que crea todas las tablas del sistema utilizando sentencias `CREATE TABLE IF NOT EXISTS`, asegurando idempotencia en los despliegues. Adicionalmente, se aplican sentencias `ALTER TABLE ADD COLUMN IF NOT EXISTS` para garantizar migraciones no destructivas. La función incluye la creación de índices sobre columnas frecuentemente consultadas (`persona_id`, `dispositivo_id`, `fecha_hora`, `empresa_id`) y la inserción de datos semilla (empresa por defecto, usuario administrador con contraseña hashada).

Conexión Wi-Fi del ESP32-CAM y gestión de conectividad

Para permitir la operación en red, el dispositivo implementa un mecanismo de conexión automática a Wi-Fi mediante la función `tryConnectWiFi()`. Esta función lee las credenciales desde `wifi.json`, configura el modo STA y deshabilita el ahorro de energía Wi-Fi (`WIFI_PS_NONE`) para mantener una conexión estable. La verificación continua de conectividad se realiza en `verificarConexionWiFi()`, que implementa un mecanismo de reintentos con backoff progresivo: tras detectar una desconexión, espera un tiempo creciente antes de reintentar (3s, 6s, 9s, 12s, 15s). Si tras 5 intentos fallidos no se logra reconectar, el dispositivo activa el Access Point como fallback.

El evento `ARDUINO_EVENT_WIFI_STA_DISCONNECTED` queda registrado con su código de razón y un contador de desconexiones, información expuesta a través del endpoint `/wifi-diag` para diagnóstico remoto.

Envío de imágenes mediante MQTT

Siguiendo el diseño establecido, la imagen es capturada, codificada en Base64 mediante una implementación optimizada que evita el uso de buffers intermedios, y enviada como un único mensaje JSON al tópico `esp32/imagen/registrar` con QoS 1 (entrega garantizada). El payload contiene dos campos: `persona_id` y `imagen` (string Base64). Este enfoque evita la complejidad de la fragmentación en múltiples mensajes y es factible porque el ESP32-CAM puede manejar cadenas de hasta 80 KB en memoria con calidad JPEG reducida (`q=10` para registro).

La función `registrarRostroEnBackend()` encapsula este flujo y retorna `true` si la publicación MQTT fue exitosa.

Recepción de imágenes en el backend vía MQTT

El backend implementa un cliente MQTT mediante la librería `paho-mqtt` en el módulo `mqtt_handler.py`. Al recibir un mensaje en `esp32/imagen/registrar`, la función `on_message` decodifica el JSON, extrae el `persona_id` y la imagen Base64, verifica que exista consentimiento biométrico registrado en la base de datos, decodifica la imagen, la guarda en `static/previews/<persona_id>.jpg` y extrae el embedding facial. Si el procesamiento es exitoso, publica en `esp32/respuesta/facial` un JSON con `status: "ok"` y el nombre del archivo; en caso de error, publica `status: "error"` con el detalle.

Adicionalmente, el cliente MQTT procesa los heartbeats (`esp32/heartbeat/<MAC>`) actualizando el estado y la IP del dispositivo en la base de datos, y los mensajes LWT (`esp32/lwt/<MAC>`) marcando el dispositivo como inactivo.

Mantenimiento de la conexión MQTT en el ESP32-CAM

La función `mantenerConexionMQTT()` se encarga de iniciar el cliente MQTT en el ESP32-CAM si el broker está configurado y hay conectividad. Soporta URLs con esquema `mqtt://`, `ws://` o `wss://` (WebSockets seguros con TLS mediante `esp_crt_bundle_attach`). Configura un mensaje LWT para que el backend sea notificado si el dispositivo se desconecta abruptamente, y se suscribe al tópico `esp32/respuesta/facial` para recibir confirmaciones del backend.

Envío de datos mediante HTTP desde el ESP32-CAM

Se implementó la capacidad del ESP32-CAM para comunicarse con la API REST del backend mediante solicitudes HTTP usando la librería `HTTPClient`. La función auxiliar `beginHttp()` configura cada petición con timeout de 10 segundos y el header `X-Device-MAC` para que el backend pueda identificar al dispositivo emisor. Esta función es utilizada por todos los handlers que requieren comunicación con el backend:

- `sincronizarPersonasDesdeBackend()`: Descarga la lista completa de personas desde `GET /api/personas` y la persiste en `personas.json`.
- `sincronizarTurnosDesdeBackend()`: Descarga turnos desde `GET /api/turnos`.
- `sincronizarAsignacionesDesdeBackend()`: Descarga asignaciones desde `GET /api/asignaciones`.
- `crearTurnoEnBackend()`: Crea un turno en el backend vía `POST /api/turnos`.
- `postAsistenciaEnBackend()`: Envía una asistencia individual vía `POST /api/asistencias`.
- `sincronizarAsistencias()`: Envía un lote de asistencias pendientes vía `POST /api/asistencias/sync`.

Implementación de la API REST del backend

El backend se implementó con Flask, organizando las rutas en blueprints independientes por dominio:

- `personas_bp (routes/personas.py)`: CRUD de personas con validación de email, unicidad de RUT y manejo de empresa.
- `turnos_bp (routes/turnos.py)`: CRUD de turnos con filtro por empresa.
- `asignaciones_bp (routes/asignaciones.py)`: CRUD de asignaciones con verificación de pertenencia a empresa.

- `asistencias_bp` (`routes/asistencias.py`): Registro de asistencias individuales y sincronización por lotes.
- `facial_bp` (`routes/facial.py`): Endpoints de reconocimiento facial (registro, verificación, identificación 1:N).
- `dispositivos_bp` (`routes/dispositivos.py`): Gestión y verificación de dispositivos.
- `logs_bp` (`routes/logs.py`): Consulta de logs de sincronización.
- `erp_bp` (`routes/erp.py`): CRUD de integraciones ERP y envío de datos.
- `auth_bp` (`routes/auth.py`): Autenticación de usuarios del panel web.

Cada blueprint se registra en la aplicación Flask principal (`app.py`), junto con la configuración de CORS para permitir peticiones desde cualquier origen. El backend se ejecuta en el puerto 5000 con modo debug activado y recarga automática deshabilitada para evitar conflictos con el cliente MQTT.

Infraestructura con Docker Compose

Para garantizar un entorno reproducible, se definió un archivo `docker-compose.yml` que levanta el broker Mosquitto (imagen `eclipse-mosquitto:latest`) con los puertos 1883 (MQTT) y 9001 (WebSockets) expuestos, volúmenes para configuración, datos y logs persistentes, y una red bridge dedicada llamada `teleasist_network`. El backend Flask y la base de datos PostgreSQL pueden ejecutarse en el host o en contenedores adicionales según la configuración de despliegue.

4.5.4 Pruebas

Se diseñaron las siguientes pruebas para validar la comunicación y el backend:

- **Prueba de conexión Wi-Fi y reconexión:** Configurar credenciales válidas en el ESP32-CAM, verificar que se conecte correctamente y obtenga una IP. Luego, apagar el router, verificar que el dispositivo detecte la desconexión mediante el evento de sistema y active el mecanismo de reintentos. Finalmente, encender el router y verificar que el dispositivo reconecte automáticamente en menos de 30 segundos.
- **Prueba de envío MQTT:** Capturar una imagen desde el ESP32-CAM y publicarla en el tópico `esp32/imagen/registrar`. Verificar que el backend reciba el mensaje, decodifique la imagen, la guarde en disco y publique una respuesta en `esp32/respuesta/facial` que el ESP32-CAM reciba correctamente.

- **Prueba de heartbeat y LWT:** Verificar que, estando el ESP32-CAM conectado, el backend reciba heartbeats periódicos (cada 30 segundos) y actualice el campo `ultimo_heartbeat` en la tabla `dispositivos`. Desconectar el dispositivo abruptamente y verificar que el mensaje LWT se publique y el backend marque el dispositivo como `inactivo` en menos de 10 segundos.
- **Prueba de recepción de datos en backend vía HTTP:** Enviar solicitudes POST a `/api/personas`, `/api/turnos`, `/api/asignaciones` y `/api/asistencias` desde el ESP32-CAM. Verificar que cada solicitud sea procesada correctamente por el backend (código HTTP 200/201) y que los datos queden persistidos en PostgreSQL.
- **Prueba de sincronización por lotes:** Generar 5 marcaciones offline en el ESP32-CAM, conectar el dispositivo a la red y ejecutar la sincronización. Verificar que los 5 registros lleguen al backend vía POST `/api/asistencias/sync`, que se inserten correctamente y que el dispositivo marque cada registro como `sincronizado = true`.
- **Prueba de inicialización de base de datos:** Ejecutar `init_db()` sobre una base de datos vacía. Verificar que se creen las 13 tablas, los índices y los datos semilla (empresa por defecto y usuario administrador) sin errores.
- **Prueba de latencia de transmisión:** Medir el tiempo total desde que el ESP32-CAM publica una imagen en MQTT hasta que recibe la respuesta de confirmación del backend. Se espera un tiempo menor a 5 segundos en condiciones de red local.

4.6 Iteración 4: Reconocimiento facial, anti-spoofing y cifrado biométrico

La cuarta iteración incorpora el procesamiento biométrico avanzado en el backend: reconocimiento facial mediante DeepFace, validación anti-spoofing para prevenir suplantaciones, y cifrado de los embeddings faciales almacenados en la base de datos. Esta etapa transforma el sistema de un simple transmisor de imágenes a un sistema con capacidad real de verificación e identificación biométrica.

4.6.1 Análisis

Para incorporar el reconocimiento facial al sistema, se evaluaron diversas librerías y modelos de *deep learning*. Se seleccionó **DeepFace** por las siguientes razones:

- Soporta múltiples modelos de reconocimiento facial (Facenet, VGG-Face, ArcFace, etc.) y detectores (RetinaFace, MTCNN, OpenCV, etc.).
- Incluye funcionalidad de **anti-spoofing** integrada, capaz de distinguir entre un rostro real y una fotografía o pantalla, analizando textura, profundidad y reflectancia de la piel.
- Su API en Python es sencilla de integrar con Flask y compatible con las imágenes JPEG capturadas por la cámara OV2640 del ESP32-CAM.
- Facenet genera embeddings de 128 dimensiones, optimizando el almacenamiento y la velocidad de comparación.

Se identificaron los siguientes requisitos funcionales para esta iteración:

- **Registro facial:** Recibir una imagen del ESP32-CAM, extraer el embedding facial, validar que el rostro no esté duplicado en la base de datos (detección de identidades repetidas) y almacenar el embedding cifrado.
- **Identificación 1:N:** Dada una imagen capturada en vivo, comparar su embedding contra todos los embeddings almacenados y retornar la identidad con mayor coincidencia, si supera el umbral de similitud.
- **Verificación 1:1:** Dada una imagen y un `persona_id`, verificar si la imagen corresponde a la persona declarada.
- **Anti-spoofing:** Detectar intentos de suplantación mediante fotografías impresas, pantallas de dispositivos o máscaras, rechazando la captura si no se detecta un rostro real.
- **Cifrado de embeddings:** Los vectores de características faciales (embeddings) constituyen datos biométricos sensibles. Para protegerlos, se deben cifrar antes de persistirlos en la base de datos, utilizando una clave derivada de una variable de entorno.
- **Consentimiento biométrico:** Todo registro facial debe estar precedido por la aceptación explícita de una política de privacidad por parte del trabajador, en cumplimiento de la normativa de protección de datos.

4.6.2 Diseño

El flujo de procesamiento facial se divide en tres operaciones principales:

1. **Registro** (POST `/api/facial/registrar`):

- Recibe `persona_id` e `imagen` (Base64).
- Verifica que exista consentimiento biométrico en la tabla `consentimientos`. Si no existe, rechaza con HTTP 403.
- Decodifica la imagen, la guarda como `static/previews/<persona_id>.jpg`.
- Extrae el embedding con DeepFace (Facenet + RetinaFace, sin anti-spoofing en esta etapa para evitar falsos rechazos por baja calidad de la cámara).
- Compara el nuevo embedding contra todos los embeddings existentes en la base de datos. Si la distancia euclidiana a algún registro es menor que 10.0, rechaza el registro con HTTP 409 (Conflicto) indicando la posible identidad duplicada.
- Si no hay duplicados, cifra el embedding con Fernet y lo almacena en la tabla `personas`.

2. Identificación 1:N (POST /api/facial/identificar):

- Recibe la imagen como JPEG crudo (`application/octet-stream`) desde el ESP32-CAM, o como JSON con Base64 desde la web.
- Guarda una copia de auditoría en `static/capturas_prueba/` con timestamp.
- Extrae el embedding con DeepFace (anti-spoofing activado mediante el parámetro `anti_spoofing=True`).
- Itera sobre todos los embeddings almacenados en la base de datos, calculando la distancia euclidiana con cada uno y seleccionando la menor.
- Si la menor distancia es inferior al umbral (10.0 para Facenet), retorna el `persona_id` correspondiente. En caso contrario, retorna HTTP 404.

3. Verificación 1:1 (POST /api/facial/verificar):

- Recibe `persona_id` e `imagen` (Base64).
- Descifra el embedding almacenado para esa persona y extrae el embedding de la imagen capturada (con anti-spoofing activado).
- Calcula la distancia euclidiana. Si es menor que 10.0, retorna éxito con el nivel de confianza calculado como $\max(0, (1 - \text{distancia}/20) * 100)$.

El modelo seleccionado es **Facenet** por su equilibrio entre precisión y tamaño de embedding, y el detector es **RetinaFace** por su robustez ante variaciones de pose e iluminación. El umbral de 10.0 para la distancia euclidiana fue elegido con base en la documentación de DeepFace y ajustes empíricos realizados durante el desarrollo.

Para el cifrado de embeddings se utiliza el algoritmo **Fernet** (AES-128 en modo CBC con HMAC-SHA256 para autenticación), parte de la librería `cryptography` de Python. La clave de cifrado se deriva aplicando SHA-256 a una variable de entorno (`BIOMETRIC_KEY`), garantizando que los embeddings nunca se almacenen en texto plano en la base de datos.

4.6.3 Implementación

Procesamiento de imágenes en el backend

El módulo `routes/facial.py` implementa los tres endpoints descritos en el diseño. Las funciones auxiliares incluyen:

- `extraer_embedding(img_path, anti_spoofing)`: Invoca `DeepFace.represent()` con el modelo Facenet y el detector RetinaFace. Si `anti_spoofing=True`, activa la detección de suplantación.
- `guardar_imagen_de_registro(persona_id, imagen_b64)`: Decodifica la imagen Base64, la convierte a RGB con Pillow y la guarda en `static/previews/<id>.jpg`.
- `_verificar_consentimiento(persona_id)`: Consulta la tabla `consentimientos` para verificar que exista un registro de aceptación.
- `_log_biometrico(persona_id, dispositivo_id, tipo_operacion, resultado)`: Inserta un registro de auditoría en `logs_biometricos`.

Procesamiento facial vía MQTT

El módulo `mqtt_handler.py` complementa los endpoints REST con un canal de procesamiento asíncrono. La función `procesar_imagen_facial(client, persona_id, imagen_b64)` implementa el mismo flujo de registro facial pero activado por la recepción de un mensaje MQTT en `esp32/imagen/registrar`. Publica el resultado en `esp32/respuesta/facial` para que el ESP32-CAM pueda reaccionar en tiempo real (confirmación con flash LED verde o error con flash rojo).

Cifrado y descifrado de embeddings

El módulo `encryption.py` proporciona dos funciones:

- `cifrar_embedding(embedding)`: Serializa el embedding como JSON, lo cifra con Fernet y codifica el resultado en Base64 URL-safe.
- `descifrar_embedding(cifrado_str)`: Decodifica el Base64, descifra con Fernet y deserializa el JSON para obtener el vector original.

Actualización y eliminación de datos faciales

Se implementó el endpoint `PUT /api/facial/actualizar/<persona_id>` que permite reemplazar el embedding y la foto de una persona existente. También se implementó `DELETE /api/personas/<id>/datos-biometricos` que elimina el embedding facial, la huella asociada, la foto en disco y el consentimiento, pero conserva los registros de asistencia históricos. La eliminación queda registrada en la tabla `eliminaciones_biometricas` para trazabilidad.

4.6.4 Pruebas

Se diseñaron las siguientes pruebas para validar el módulo de reconocimiento facial:

- **Prueba de registro facial exitoso:** Enviar una imagen facial válida junto con un `persona_id` que tenga consentimiento registrado. Verificar que el backend extraiga el embedding, lo almacene cifrado y responda con HTTP 200. Verificar que la imagen se guarde en `static/previews/`.
- **Prueba de detección de duplicados:** Registrar una segunda imagen del mismo rostro para otra persona. Verificar que el backend detecte la similitud (distancia < 10.0) y rechace el registro con HTTP 409, indicando el nombre de la persona con la que coincide.
- **Prueba de rechazo por falta de consentimiento:** Intentar registrar un rostro para una persona sin consentimiento biométrico. Verificar que el backend rechace la operación con HTTP 403.
- **Prueba de identificación 1:N:** Registrar 3 personas con sus respectivas fotos. Enviar una nueva foto de una de ellas al endpoint de identificación. Verificar que el backend retorne el `persona_id` correcto y que la copia de auditoría se guarde en `static/capturas_prueba/`.
- **Prueba de identificación con rostro desconocido:** Enviar una foto de una persona no registrada. Verificar que el backend retorne HTTP 404 y registre el intento fallido en `logs_biometricos`.
- **Prueba de anti-spoofing con fotografía impresa:** Capturar una foto de un rostro registrado desde una pantalla o impresión y enviarla al endpoint de identificación. Verificar que DeepFace con `anti_spoofing=True` rechace la imagen por no detectar un rostro real (excepción `ValueError` con mensaje “Spoof detected”).

- **Prueba de resistencia a condiciones de iluminación:** Capturar imágenes del mismo rostro bajo 3 condiciones de iluminación (luz natural, luz artificial tenue, contraluz). Verificar que los 3 embeddings generados mantengan distancia euclidiana entre sí menor a 10.0 y que el sistema identifique correctamente a la persona.
- **Prueba de cifrado y descifrado:** Almacenar un embedding en la base de datos y verificar que el valor en la columna `encoding_facial` no sea legible (no contenga los números del vector en texto plano). Luego, recuperar el registro y descifrar el embedding; verificar que el vector original se reconstruya idénticamente.
- **Prueba de eliminación de datos biométricos:** Ejecutar el endpoint de eliminación de datos biométricos para una persona con rostro registrado. Verificar que el embedding se elimine de la base de datos, que la foto se borre del disco, que el consentimiento se elimine, que los registros de asistencia se conserven y que quede una entrada en `eliminaciones_biometricas`.

4.7 Iteración 5: Autenticación, multi-tenant y enrolamiento de dispositivos

La quinta iteración incorpora la capa de seguridad y administración del sistema: autenticación de usuarios del panel web mediante JWT, un modelo multi-tenant que permite a múltiples empresas operar sobre la misma instancia del backend con datos aislados, y un mecanismo seguro de enrolamiento de dispositivos ESP32-CAM mediante PIN de un solo uso.

4.7.1 Análisis

El sistema, al estar orientado a un entorno empresarial, requiere que diferentes organizaciones puedan utilizar la misma plataforma sin que sus datos se mezclen. Asimismo, el panel web administrativo debe restringir el acceso según el rol del usuario. Finalmente, cada dispositivo ESP32-CAM debe ser vinculado de forma segura a una empresa específica, evitando que dispositivos no autorizados inyecten datos en el sistema.

Los requisitos identificados son:

- **Multi-tenant:** Cada entidad (persona, turno, asignación, asistencia, dispositivo) debe pertenecer a una empresa. Las consultas deben filtrarse por empresa según el contexto del usuario autenticado. El administrador global (rol `admin`) tiene visibilidad sobre todas las empresas.

- **Autenticación JWT:** El panel web backend debe requerir autenticación para operaciones sensibles. Los tokens JWT (JSON Web Tokens), firmados con HMAC-SHA256, transportan el ID de usuario, la empresa seleccionada y su rol. Tienen expiración de 24 horas.
- **Roles de usuario:** Se definen tres roles jerárquicos:
 - **admin:** Acceso total al sistema, puede crear empresas, gestionar todos los usuarios y dispositivos.
 - **empleador:** Acceso limitado a los datos de su empresa; puede gestionar trabajadores, turnos y ver asistencias.
 - **trabajador:** Acceso restringido a sus propios datos de asistencia.
- **Enrolamiento de dispositivos:** Un administrador genera un PIN de 8 caracteres en el panel web. Ese PIN se ingresa en el ESP32-CAM (vía web embebida), que lo envía al backend junto con su MAC address. Si el PIN es válido y no ha sido usado, el dispositivo queda enrolado y asociado a la empresa correspondiente.
- **Monitorización de dispositivos:** Una vez enrolado, el ESP32-CAM envía heartbeats periódicos vía MQTT. El backend actualiza el estado del dispositivo (**activo**) y registra su última actividad. Un proceso watchdog marca como **inactivo** a los dispositivos que no han enviado heartbeat en más de 90 segundos.

4.7.2 Diseño

Modelo de datos multi-tenant

La estrategia multi-tenant se implementa a nivel de base de datos mediante una columna `empresa_id` en las tablas `personas`, `turnos`, `dispositivos`, `integraciones_erp` y `asistencias` (esta última a través de la relación con `personas`). La tabla `empresas` actúa como entidad raíz del tenant. La tabla asociativa `usuario_empresa` vincula usuarios con empresas y define el rol que cada usuario tiene en cada empresa, permitiendo que un mismo usuario tenga roles distintos en diferentes organizaciones.

Flujo de autenticación

1. El usuario envía email y contraseña al endpoint `POST /api/auth/login`.
2. El backend valida las credenciales contra la tabla `usuarios_web` usando `bcrypt`.
3. Se consultan las empresas asociadas al usuario en `usuario_empresa`.

4. Si el usuario pertenece a una sola empresa, se selecciona automáticamente. Si pertenece a varias, se retorna la lista para que elija.
5. Se genera un token JWT con payload: `user_id`, `empresa_id`, `rol`, `persona_id` (si es trabajador) y `exp` (24 horas).
6. Las peticiones subsecuentes deben incluir el header `Authorization: Bearer <token>`.

Se implementan dos niveles de protección para los endpoints:

- `@token_required`: Exige token JWT válido.
- `@requiere_rol('admin', 'empleador')`: Además del token, exige que el rol del usuario esté en la lista permitida.
- `@token_opcional`: Intenta extraer el token si está presente; si no, permite el acceso sin autenticación (útil para endpoints consumidos por el ESP32-CAM, que en su lugar envían el header `X-Device-MAC` para identificar la empresa).

Flujo de enrolamiento de dispositivos

1. Un administrador o empleador accede a `POST /api/dispositivos/generar-pin` desde el panel web, especificando un nombre para el dispositivo. El backend genera un PIN alfanumérico de 8 caracteres (ej. `A3B9XK2M`) usando `secrets.choice()` y lo inserta en la tabla `dispositivos` con `enrolado = FALSE`.
2. El administrador ingresa el PIN en la interfaz web del ESP32-CAM (`/wifi-setup`).
3. Al guardar la configuración, el ESP32-CAM envía `POST /api/dispositivos/enrolar` con el PIN, su MAC address y su IP local.
4. El backend valida el PIN, asocia la MAC y la IP al dispositivo, marca `enrolado = TRUE` y limpia el PIN para que no pueda ser reutilizado.
5. A partir de este momento, el ESP32-CAM envía heartbeats periódicos y el backend lo reconoce como dispositivo autorizado de la empresa.

Heartbeat y Last Will Testament (LWT)

El ESP32-CAM publica periódicamente (cada 30 segundos) un mensaje JSON en el tópico `esp32/heartbeat/<MAC>` con su MAC, timestamp e IP. El backend actualiza el campo `ultimo_heartbeat` y marca el estado como `activo`. Al conectarse al broker MQTT, el ESP32-CAM configura un mensaje LWT en el tópico `esp32/lwt/<MAC>` con

`{"estado": "inactivo"}`. Si el dispositivo se desconecta abruptamente, el broker publica automáticamente el LWT, y el backend marca el dispositivo como `inactivo`. Adicionalmente, un hilo `device_watchdog` ejecutado cada 60 segundos verifica qué dispositivos no han enviado heartbeat en más de 90 segundos y los marca como inactivos.

4.7.3 Implementación

Módulo de autenticación

El módulo `routes/auth.py` implementa:

- `login()`: Valida credenciales con `bcrypt`, determina la empresa (con selección múltiple si aplica), vincula al trabajador con su `persona_id` si el rol es `trabajador`, y genera el token JWT.
- `register()`: Permite a administradores y empleadores crear nuevos usuarios, asignándoles rol y empresa. Si el email ya existe, se agrega la nueva asociación `usuario_empresa` en lugar de duplicar el usuario.
- `me()`: Retorna los datos del usuario autenticado, incluyendo sus empresas y rol actual.
- `cambiar_password()`: Permite al usuario cambiar su contraseña validando la actual.
- `generar_pin_enrolamiento()`: Crea un PIN para enrolar un nuevo dispositivo.
- `enrolar_dispositivo()`: Valida el PIN y completa el enrolamiento del ESP32-CAM.
- `listar_usuarios()`, `eliminar_usuario()`, `listar_empresas()`, `crear_empresa()`: Operaciones administrativas con control de roles.

Los decoradores `@token_required`, `@token_opcional` y `@requiere_rol` se definen en el mismo módulo. El decorador `@token_opcional` incluye lógica para inferir la empresa a partir del header `X-Device-MAC` cuando no hay token JWT presente, buscando la MAC en la tabla de dispositivos.

Gestión de dispositivos

El módulo `routes/dispositivos.py` implementa:

- `GET /api/dispositivos`: Lista los dispositivos con su estado, MAC, IP y empresa.

- **PUT /api/dispositivos/<id>:** Actualiza el nombre del dispositivo.
- **DELETE /api/dispositivos/<id>:** Elimina un dispositivo (admin) o lo desvincula de la empresa (empleador).
- **POST /api/dispositivos/verificar:** Prueba conectividad con un dispositivo dada su IP, consultando el endpoint /estado del ESP32-CAM.

Monitorización vía MQTT

En el módulo `mqtt_handler.py`, la función `on_message` maneja los tópicos de heartbeat y LWT:

- **esp32/heartbeat/<MAC>:** Actualiza `ultimo_heartbeat`, `estado = 'activo'` y opcionalmente `ip_local` del dispositivo.
- **esp32/lwt/<MAC>:** Marca `estado = 'inactivo'` en la base de datos.

El watchdog (`device_watchdog`) ejecuta un barrido inicial al arrancar (todos los dispositivos se marcan inactivos hasta que envíen su primer heartbeat) y luego verifica cada 60 segundos los heartbeats vencidos.

4.7.4 Pruebas

Se diseñaron las siguientes pruebas para validar la autenticación, el multi-tenant y el enrolamiento:

- **Prueba de login exitoso:** Enviar credenciales válidas del administrador por defecto (`admin@empresa.cl / admin123`). Verificar que se reciba un token JWT, los datos del usuario y su empresa. Decodificar el token y verificar que contenga `user_id`, `empresa_id` y `rol = admin`.
- **Prueba de login con múltiples empresas:** Crear una segunda empresa y asignar el mismo usuario a ella con rol `empleador`. Iniciar sesión sin especificar empresa. Verificar que el backend retorne la lista de empresas disponibles. Luego, iniciar sesión especificando una empresa y verificar que el token refleje esa selección.
- **Prueba de rechazo de credenciales inválidas:** Intentar login con contraseña incorrecta. Verificar HTTP 401 y mensaje “Credenciales inválidas”.
- **Prueba de acceso sin token:** Intentar acceder a `GET /api/personas` sin enviar header `Authorization`, desde un cliente sin X-Device-MAC. Verificar que el endpoint responda correctamente (acceso público o con datos limitados según el decorador).

- **Prueba de acceso con rol insuficiente:** Autenticarse como trabajador e intentar crear un turno (POST /api/turnos). Verificar HTTP 403 y mensaje “Permisos insuficientes”.
- **Prueba de aislamiento multi-tenant:** Crear dos empresas (A y B), registrar una persona en cada una. Autenticarse como empleador de la empresa A y solicitar GET /api/personas. Verificar que solo se retornen las personas de la empresa A.
- **Prueba de generación de PIN:** Autenticarse como empleador y solicitar POST /api/dispositivos/generar-pin. Verificar que se reciba un PIN de 8 caracteres alfanuméricos y que en la base de datos exista un dispositivo con `enrolado = FALSE`.
- **Prueba de enrolamiento exitoso:** Con un PIN generado, enviar POST /api/dispositivos/enrolar con el PIN, una MAC y una IP. Verificar que el backend marque el dispositivo como enrolado, asocie la MAC y limpie el PIN. Verificar que el mismo PIN no pueda usarse una segunda vez (HTTP 409).
- **Prueba de heartbeat y estado:** Simular heartbeats desde un dispositivo enrolado. Verificar que el estado en la base de datos sea `activo`. Detener los heartbeats por más de 90 segundos y verificar que el watchdog marque el dispositivo como `inactivo`.
- **Prueba de LWT:** Conectar el ESP32-CAM al broker MQTT, verificar que el backend reciba el heartbeat y lo marque activo. Desconectar el dispositivo abruptamente (corte de energía). Verificar que el broker publique el LWT y el backend actualice el estado a `inactivo` en menos de 10 segundos.

4.8 Iteración 6: Módulo antifraude y validación previa a la captura

La sexta iteración aborda la prevención de suplantación de identidad en el dispositivo ESP32-CAM mediante una estrategia combinada de detección física de presencia, control de iluminación y procesamiento biométrico. Si bien el plan original contemplaba un sensor infrarrojo (IR) dedicado para detectar intentos de suplantación con fotografías, el enfoque implementado evolucionó hacia una solución de múltiples capas que resultó más robusta y prescindió del sensor IR adicional.

4.8.1 Análisis

El análisis de escenarios de suplantación identificó los siguientes vectores de ataque potenciales contra el sistema de reconocimiento facial del ESP32-CAM:

- **Fotografía impresa:** Un atacante presenta una foto del rostro de un trabajador legítimo frente a la cámara.
- **Pantalla de dispositivo:** Un atacante muestra un video o imagen del trabajador en la pantalla de un teléfono o tablet.
- **Ausencia de presencia real:** La cámara captura sin que haya una persona físicamente frente al dispositivo (falsos positivos del sensor de movimiento).

Inicialmente se planificó el uso de un sensor IR dedicado que permitiera detectar la presencia de un rostro real mediante la emisión y recepción de luz infrarroja, cuya reflectancia difiere entre piel humana y superficies planas como papel o pantallas. Sin embargo, durante el desarrollo se identificaron dos factores que motivaron el cambio de estrategia:

1. **Disponibilidad de GPIO:** El ESP32-CAM tiene una cantidad muy limitada de pines GPIO disponibles tras la asignación del bus paralelo de la cámara. Destinar un pin adicional al sensor IR hubiera requerido sacrificar otra funcionalidad (como el sensor PIR de presencia o el control PWM del flash).
2. **Madurez del anti-spoofing por software:** DeepFace, la librería seleccionada para el reconocimiento facial, incorpora un detector de suplantación (*anti-spoofing*) basado en redes neuronales convolucionales que analiza micro-texturas, reflectancia de la piel y profundidad en la imagen capturada. Este detector es capaz de distinguir entre un rostro real tridimensional y una superficie plana (foto, pantalla) con una precisión comparable o superior a la de sensores IR de bajo costo como los inicialmente considerados.

En consecuencia, se adoptó un enfoque de múltiples capas que combina:

1. **Detección física de presencia (PIR):** Un sensor infrarrojo pasivo (PIR) que detecta el calor corporal, activando el sistema de captura solo cuando hay una persona real frente al dispositivo. Esto elimina falsos positivos del software y reduce el consumo energético del sensor de huella y la cámara.
2. **Iluminación controlada (flash PWM):** Un destello breve y de intensidad moderada (50% de duty cycle a 5 kHz) que ilumina el rostro durante la captura

sin encenegecer a la persona. La respuesta del rostro real a esta fuente de luz puntual difiere de la de una superficie plana, aportando una capa adicional de discriminación.

3. **Anti-spoofing por software (DeepFace):** La capa más potente, que analiza la imagen capturada con un modelo entrenado para detectar intentos de suplantación. Las imágenes que no superan esta validación son rechazadas con una excepción `ValueError` que el backend traduce en un mensaje de error al ESP32-CAM.

4.8.2 Diseño

Sensor PIR como portero del sistema

El PIR conectado al GPIO12 actúa como “portero”: el sistema permanece en reposo hasta que el sensor detecta movimiento (cambio en la radiación infrarroja del entorno). Una vez activado, se inicia un período de “alerta” de 15 segundos durante el cual:

- Se habilita el escaneo de huella cada 2 segundos mediante el AS608.
- Si hay conectividad, se habilita la identificación facial cada 6 segundos.
- Si el PIR no vuelve a detectar movimiento en 15 segundos, se asume que fue una falsa alarma o que la persona se retiró, y el sistema vuelve a reposo.

Esta arquitectura reduce drásticamente el consumo energético y el desgaste del sensor de huellas, ya que el AS608 solo se activa cuando hay una alta probabilidad de que haya una persona frente al dispositivo.

Control de acceso con cooldown y bloqueo

Para evitar marcaciones múltiples accidentales y garantizar que cada interacción sea deliberada, se implementan tres mecanismos de control:

1. **Cooldown entre marcaciones** (`COOLDOWN_TIEMPO = 8000 ms`): Tras una marcación exitosa, el sistema ignora nuevos intentos durante 8 segundos.
2. **Debounce de huella** (`FINGER_DEBOUNCE = 4000 ms`): Una misma huella no puede gatillar dos marcaciones en un intervalo menor a 4 segundos, evitando lecturas múltiples del mismo dedo.
3. **Bloqueo por menú** (`BLOQUEO_MENU_MS = 30000 ms`): Cuando se utiliza la interfaz web para operaciones administrativas (registro, edición, configuración), se activa un bloqueo de 30 segundos que impide la marcación automática, evitando que la persona que está operando el menú sea registrada inadvertidamente.

Firma de movimiento

Para detectar si la escena frente a la cámara cambió entre dos capturas consecutivas (lo que indicaría una persona real moviéndose versus una foto estática), se implementa una **firma de movimiento** que calcula un hash reducido del buffer JPEG:

- Se muestrean 128 bytes del frame capturado, distribuidos uniformemente a lo largo del buffer.
- Se calcula la suma acumulada de esos bytes, produciendo una firma de 32 bits.
- Si la diferencia entre la firma actual y la anterior supera un umbral (`UMBRAL_MOVIMIENTO = 1800`), se considera que hubo movimiento real en la escena.

4.8.3 Implementación

Integración del PIR

El sensor PIR se conecta al GPIO12 del ESP32-CAM, configurado con resistencia de pull-down interna para evitar lecturas flotantes. En el `setup()` se incluye una fase de calibración de 3 segundos (`delay(3000)`) para estabilizar el sensor antes de comenzar el bucle principal.

En el `loop()`, la lógica de detección sigue el siguiente flujo:

- Si el PIR detecta `HIGH` y el flag `hayAlguienFrenteAlSensor` es falso, se activa el modo alerta y se esperan 800 ms para estabilizar la fuente de alimentación (el capacitor de 1000 μ F del ESP32-CAM necesita recuperarse antes de encender la cámara).
- Si el PIR detecta `HIGH` estando ya en modo alerta, se renueva el timer de actividad.
- Si pasan más de 15 segundos sin detección, se desactiva el modo alerta y se retorna a reposo.

Flash PWM para iluminación facial

El flash LED integrado del ESP32-CAM se controla mediante PWM en el GPIO4, configurado como salida LED PWM con frecuencia de 5 kHz y resolución de 8 bits. Durante la captura de una imagen (tanto para registro como para identificación), el flash se activa con un duty cycle del 50% (`FLASH_PWM_DUTY_50 = 128`) durante 150–200 milisegundos, tiempo suficiente para que el sensor OV2640 ajuste su exposición.

La secuencia de captura es la siguiente:

1. Encender flash al 50% (`ledcWrite(FLASH_PIN, 128)`).

2. Esperar 150 ms para estabilización del sensor.
3. Capturar el frame (`esp_camera_fb_get()`).
4. Apagar el flash inmediatamente (`ledcWrite(FLASH_PIN, 0)`).
5. Procesar o transmitir el frame.

Este enfoque minimiza el tiempo total de encendido del flash y, por tanto, el consumo eléctrico y la molestia para el usuario.

Señalización mediante flash

Además de la iluminación para captura, el flash se utiliza para señalar el resultado de las operaciones:

- **Éxito** (`flashExito()`): Dos destellos breves (150 ms encendido, 150 ms apagado).
- **Error** (`flashError()`): Un destello largo (800 ms encendido).

Esta señalización permite al usuario recibir retroalimentación inmediata sin necesidad de mirar una pantalla.

Validación anti-spoofing en el backend

La validación anti-spoofing se ejecuta en el backend, no en el ESP32-CAM, debido a las limitaciones de procesamiento del microcontrolador. La función `extraer_embedding()` en `routes/facial.py` recibe el parámetro `anti_spoofing=True` para los endpoints de identificación y verificación, lo que activa el detector de suplantación de DeepFace. Si la imagen no supera la validación (por ser una foto, pantalla o carecer de características faciales reales), DeepFace lanza una excepción `ValueError` que el backend captura y traduce en una respuesta HTTP 400 con el mensaje “Spoof detected”.

4.8.4 Pruebas

Se diseñaron las siguientes pruebas para validar el módulo antifraude:

- **Prueba de detección PIR:** Situar a una persona a 50 cm del dispositivo. Verificar que el PIR detecte el movimiento en menos de 1 segundo y que el sistema transicione a modo activo. Retirar a la persona y verificar que tras 15 segundos sin detección, el sistema retorne a reposo.

- **Prueba de falsos positivos del PIR:** Dejar el dispositivo en una habitación vacía durante 10 minutos y monitorear los logs. Verificar que no se active el modo de captura sin presencia humana real (tolerancia máxima: 1 falso positivo por hora).
- **Prueba de cooldown:** Realizar una marcación exitosa por huella. Intentar inmediatamente una segunda marcación. Verificar que el sistema rechace el intento por estar en período de cooldown (8 segundos).
- **Prueba de debounce de huella:** Mantener el dedo presionado sobre el sensor AS608 durante 5 segundos. Verificar que solo se registre una marcación, no múltiples.
- **Prueba de bloqueo por menú:** Acceder a la vista `/register` desde el navegador. Mientras la página está abierta, intentar provocar una marcación automática con huella. Verificar que el sistema esté bloqueado durante 30 segundos.
- **Prueba de anti-spoofing con fotografía:** Imprimir una foto de un rostro registrado y presentarla frente al ESP32-CAM con el PIR activado (para ello, mover la foto simulando una persona). Verificar que DeepFace rechace la imagen con `anti_spoofing=True` y que el backend retorne HTTP 400.
- **Prueba de anti-spoofing con pantalla:** Reproducir un video del rostro de una persona registrada en un teléfono móvil y presentarlo frente al ESP32-CAM. Verificar que el sistema rechace la identificación.
- **Prueba de flash PWM:** Durante una captura, medir con un osciloscopio o cámara de alta velocidad que el flash se encienda al 50% de duty cycle durante no más de 200 ms. Verificar que la iluminación sea suficiente para obtener una imagen nítida del rostro.
- **Prueba de señalización:** Provocar una marcación exitosa y verificar que el flash emita dos destellos breves. Provocar un error (huella no reconocida) y verificar que el flash emita un destello largo.

4.9 Iteración 7: Panel web backend e integración con ERP

La séptima iteración incorpora el panel de administración web del backend y el módulo de integración con sistemas ERP externos, permitiendo que los datos de asistencia

capturados por el dispositivo IoT sean automáticamente transmitidos a plataformas empresariales de recursos humanos o nómina.

4.9.1 Análisis

El sistema, hasta esta iteración, permite registrar asistencias y almacenarlas en la base de datos, pero carece de una interfaz administrativa centralizada y de un mecanismo para integrar los datos con sistemas ERP existentes. Se identificaron los siguientes requisitos:

- **Panel web administrativo:** Una interfaz web que permita a administradores y empleadores gestionar personas, turnos, asignaciones, visualizar asistencias, administrar dispositivos enrolados y configurar integraciones ERP. Debe ser accesible vía navegador sin dependencia del ESP32-CAM.
- **API REST completa y documentada:** Todos los blueprints implementados en iteraciones anteriores deben consolidarse como una API REST coherente, con métodos HTTP estándar, respuestas JSON estructuradas y manejo de autenticación por JWT.
- **Integración con ERP vía webhooks:** Permitir configurar múltiples destinos ERP, cada uno con su propia URL de webhook, headers personalizados (ej. tokens de autenticación) y mapeo de campos (field mapping) para adaptar el formato de los datos de asistencia al esquema esperado por cada ERP.
- **Envío automático:** Las asistencias registradas deben enviarse automáticamente a los ERP configurados, en un hilo separado para no bloquear la respuesta HTTP al dispositivo.
- **Test de conectividad:** Cada integración ERP debe poder probarse manualmente desde el panel web antes de activar el envío automático.

4.9.2 Diseño

Arquitectura del backend

El backend se organiza en 9 blueprints de Flask, cada uno con responsabilidades bien definidas:

1. **auth_bp:** Autenticación JWT, registro de usuarios, gestión de empresas, enrolamiento de dispositivos.

2. `personas_bp`: CRUD de personas con consentimiento biométrico y eliminación de datos sensibles.
3. `turnos_bp`: CRUD de turnos con filtro por empresa.
4. `asignaciones_bp`: CRUD de asignaciones persona-turno.
5. `asistencias_bp`: Registro de asistencias individuales y sincronización por lotes.
6. `facial_bp`: Registro, verificación e identificación facial con anti-spoofing y cifrado.
7. `dispositivos_bp`: Gestión de dispositivos enrolados y verificación de conectividad.
8. `logs_bp`: Consulta de logs de sincronización y logs biométricos.
9. `erp_bp`: CRUD de integraciones ERP, envío de datos y test de conectividad.

Diseño del módulo ERP

La integración con ERP se basa en el patrón **webhook saliente**: cuando se registra una asistencia, el backend notifica a cada ERP configurado mediante una solicitud HTTP POST al webhook correspondiente, con los datos de la marcación en el cuerpo JSON.

Cada integración ERP se configura con los siguientes parámetros:

- **nombre**: Identificador descriptivo (ej. “Softland RRHH”).
- **tipo**: Clasificación del ERP (`generic`, `softland`, `defontana`, `sap`).
- **webhook_url**: Endpoint HTTP donde el ERP espera recibir los datos.
- **headers**: Objeto JSON con headers HTTP personalizados (ej. `{"Authorization": "Bearer <token>"}`).
- **field_map**: Objeto JSON que mapea los campos estándar del sistema (`persona_id`, `nombre`, `tipo`, `metodo`, `fecha_hora`, `rut`) a los nombres de campo esperados por el ERP.
- **envio_auto**: Booleano que indica si el envío se realiza automáticamente al registrar cada asistencia, o solo manualmente.
- **activo**: Booleano para habilitar o deshabilitar temporalmente la integración sin eliminarla.

El field mapping permite adaptar el formato de salida sin modificar el código del backend. Por ejemplo, un ERP que espera los campos `employee_id`, `timestamp` y `event_type` puede configurarse con:

```
{"persona_id": "employee_id", "fecha_hora": "timestamp", "tipo": "event_type"}
```

4.9.3 Implementación

Panel web del backend

Para complementar el backend, se desarrolló un panel web independiente (Next.js 16 con React 19 y TypeScript) cuyo módulo principal es la gestión del dispositivo IoT. Este panel, accesible desde cualquier navegador moderno, permite al administrador:

- Visualizar el estado online/offline de cada dispositivo enrolado mediante tarjetas con código de colores.
- Generar el PIN de 8 caracteres necesario para enrolar un nuevo ESP32-CAM.
- Probar la conectividad activa con un dispositivo específico consultando su endpoint `/estado` (variable `NEXT_PUBLIC_DEVICE_BASE_URL`, por defecto `http://192.168.4.1`).
- Renombrar y eliminar dispositivos registrados.
- Acceder a los logs de sincronización generados por el dispositivo.
- Acceder a las funcionalidades complementarias de gestión: personas, turnos, asignaciones, asistencias, integración ERP, usuarios y empresas.

El panel web se comunica con el backend Flask mediante un proxy interno (`app/api/_proxy.ts`) que reenvía las peticiones a la API REST, evitando problemas de CORS. La aplicación está protegida por un middleware de autenticación que valida el token JWT y restringe el acceso según el rol del usuario (admin, empleador, trabajador). El frontend se compila con Docker en un build multi-etapa usando `output: 'standalone'` para producir una imagen optimizada lista para producción.

Endpoint de configuración ERP para el ESP32-CAM

Para que el ESP32-CAM pueda notificar asistencias directamente a los ERP configurados (incluso si el backend central no está disponible), se implementó el endpoint `GET /api/dispositivos/erp-config`, que retorna la lista de integraciones ERP activas con envío automático habilitado. El ESP32-CAM consulta este endpoint cada hora y almacena la configuración en `/erp-config.json`, permitiéndole enviar marcaciones a los ERP incluso durante desconexiones parciales.

Módulo de integración ERP

El módulo `routes/erp.py` implementa:

- **enviar_asistencia_a_erps()**: Función central que itera sobre las integraciones ERP activas de una empresa, transforma los datos de la asistencia según el field mapping configurado, y envía una solicitud POST al webhook con los headers personalizados. Registra el resultado (`ok` o `error`) en la tabla `integraciones_erp` (columnas `ultimo_envio` y `ultimo_estado`).
- **_disparar_erp_push()**: Wrapper que ejecuta `enviar_asistencia_a_erps()` en un hilo daemon separado, evitando que la latencia de los ERP externos afecte el tiempo de respuesta al ESP32-CAM. Esta función se invoca automáticamente desde `create_asistencia()` y `sync_asistencias()` cada vez que se registra una asistencia.
- **CRUD de integraciones**: Endpoints REST para crear, listar y eliminar configuraciones ERP.
- **Test de conectividad** (POST `/api/erp/<id>/test`): Envía un payload de prueba con datos ficticios al webhook configurado y retorna el código de respuesta y el cuerpo recibido.
- **Envío manual** (POST `/api/erp/<id>/enviar`): Envía las últimas 200 asistencias de la empresa al ERP seleccionado, útil para cargas históricas o reprocesamiento.
- **Estado de integración** (GET `/api/erp/<id>/estado`): Retorna el último envío y su estado.

4.9.4 Pruebas

Se diseñaron las siguientes pruebas para validar el panel web para la gestión del dispositivo y la integración ERP:

- **Prueba de navegación en panel web**: Acceder a las rutas principales del backend (`/health`, `/api/personas`, `/api/asistencias`) sin autenticación y verificar que respondan correctamente (HTTP 200 con acceso público o HTTP 401 cuando se requiera token).
- **Prueba de creación de integración ERP**: Autenticarse como empleador y enviar POST `/api/erp` con nombre, tipo, webhook URL (usando un servidor de prueba como `https://webhook.site`) y field mapping. Verificar que la integración se cree con `activo = TRUE` y `envio_auto = TRUE`.

- **Prueba de test de webhook:** Ejecutar `POST /api/erp/<id>/test` para la integración creada. Verificar que el webhook de prueba reciba un payload JSON con datos ficticios (`persona_id = 99`, `nombre = "Test Usuario"`, `tipo = "entrada"`). Verificar que el estado se actualice en la base de datos.
- **Prueba de envío automático:** Registrar una asistencia real (`POST a /api/asistencias`) para una persona de la empresa configurada. Verificar que el servidor de prueba del webhook reciba automáticamente el payload con los datos de la asistencia.
- **Prueba de field mapping:** Configurar un ERP con field mapping `{"persona_id": "employee_id", "tipo": "event"}`. Registrar una asistencia y verificar que el webhook reciba el payload con los campos renombrados (`employee_id` en lugar de `persona_id`, `event` en lugar de `tipo`).
- **Prueba de envío asíncrono:** Medir el tiempo de respuesta del endpoint `POST /api/asistencias` cuando hay un ERP configurado cuyo webhook tarda 5 segundos en responder. Verificar que la respuesta al cliente sea inmediata (menor a 200 ms) y que el envío al ERP ocurra en segundo plano.
- **Prueba de tolerancia a fallos ERP:** Configurar un webhook que retorne `HTTP 500`. Registrar una asistencia y verificar que el sistema no falle; el error debe quedar registrado en `ultimo_estado` de la integración y la asistencia debe persistirse correctamente en la base de datos.
- **Prueba de sincronización de configuración ERP al ESP32-CAM:** Simular la consulta `GET /api/dispositivos/erp-config` desde el ESP32-CAM (con header `X-Device-MAC`). Verificar que retorne solo las integraciones activas con envío automático de la empresa correspondiente.
- **Prueba de envío manual por lote:** Ejecutar `POST /api/erp/<id>/enviar` y verificar que se envíen correctamente los registros de asistencia recientes al webhook del ERP.

4.10 Iteración 8: Sincronización diferida, logs y cierre del proyecto

La octava y última iteración integra los mecanismos de sincronización diferida entre el ESP32-CAM y el backend, implementa la capa de auditoría y logs del sistema, incorpora herramientas de diagnóstico, y consolida el prototipo como un sistema completo y funcional.

4.10.1 Análisis

Para que el prototipo sea viable en entornos reales, debe garantizar que ningún dato se pierda durante períodos de desconexión y que todas las operaciones críticas queden registradas para auditoría. Los requisitos identificados son:

- **Sincronización diferida de asistencias:** Los registros de asistencia generados offline (marcados con `sincronizado = false`) deben enviarse automáticamente al backend cuando se restablezca la conectividad. El proceso debe ser idempotente: si un registro ya existe en el backend, no debe duplicarse.
- **Sincronización diferida de entidades:** Las personas, turnos y asignaciones creados localmente durante el modo offline deben sincronizarse con el backend, resolviendo conflictos de identificadores (IDs locales con prefijo `local-` versus IDs del backend).
- **Sincronización de configuración ERP:** La configuración de integraciones ERP debe poder descargarse desde el backend para que el ESP32-CAM pueda enviar datos de asistencia directamente a los ERP configurados.
- **Logs de sincronización:** Cada operación de sincronización debe registrarse en la base de datos con el conteo de registros enviados y procesados correctamente, permitiendo auditoría posterior.
- **Logs biométricos:** Todas las operaciones de registro, verificación e identificación facial deben quedar registradas con timestamp, resultado e IP de origen, para cumplimiento normativo.
- **Watchdog de dispositivos:** El backend debe monitorear continuamente el estado de los dispositivos enrolados, detectando aquellos que han dejado de comunicarse y marcándolos como inactivos.
- **Herramientas de diagnóstico:** Se requiere un script de prueba que permita simular asistencias faciales usando fotos almacenadas, facilitando las pruebas del sistema sin necesidad del hardware.

4.10.2 Diseño

Proceso de sincronización diferida

El ESP32-CAM ejecuta la sincronización en dos momentos:

1. **Al iniciar:** Si hay conectividad, llama a `sincronizarPendientes()` que ejecuta en secuencia:

- `sincronizarTurnosPendientes()`: Envía los turnos con `sincronizado = false` al backend vía POST, y actualiza sus `backend_id`.
- `sincronizarAsignacionesPendientes()`: Similar para asignaciones, dependiendo de que los turnos referenciados ya tengan `backend_id`.
- `sincronizarAsistencias()`: Envía un lote de asistencias con `sincronizado = false` vía POST `/api/asistencias/sync`.

2. **Periódicamente:** Cada 5 minutos (300,000 ms), se ejecuta `sincronizarPendientes()` en el bucle principal.

Además, el ESP32-CAM consulta `GET /api/dispositivos/erp-config` cada hora (360,000 ms) para mantener actualizada su configuración local de webhooks ERP.

Deduplicación en el backend

El endpoint `POST /api/asistencias/sync` implementa una verificación previa a la inserción: para cada registro del lote, consulta si ya existe una asistencia de la misma persona con el mismo tipo en una ventana de 60 segundos. Si existe, omite la inserción. Esto garantiza idempotencia ante reenvíos.

Tablas de auditoría

- `sincronizacion_log`: Registra cada operación de sincronización con: dispositivo de origen, cantidad de registros enviados, cantidad procesada correctamente, estado y detalle.
- `logs_biometricos`: Registra cada operación biométrica con: persona, dispositivo, timestamp, tipo de operación (registro, verificación, identificación), resultado (éxito, fallo, duplicado, no_encontrado) e IP de origen.
- `eliminaciones_biometricas`: Registra cada eliminación de datos biométricos con: persona, embedding anterior (cifrado), ruta de la foto eliminada y usuario solicitante.

Panel de visualización de logs

El backend expone dos conjuntos de endpoints para consulta de logs:

- `GET /api/logs`: Lista los registros de sincronización, con filtro por empresa para roles no-admin.
- `DELETE /api/logs`: Permite limpiar los logs (admin limpia todo, empleador limpia los de su empresa).

- El ESP32-CAM expone `GET /api/logs` que retorna el buffer de logs en formato HTML, y `/api/logs/clear` para limpiarlo.

Script de simulación de asistencia facial

Se desarrolló `deteccion.py`, un script de línea de comandos que permite probar el flujo de reconocimiento facial sin necesidad del ESP32-CAM. El script escanea la carpeta `static/capturas_prueba/`, presenta un menú interactivo para seleccionar una foto, ejecuta el proceso de identificación 1:N contra la base de datos y registra la asistencia correspondiente. Es útil para pruebas de regresión rápida y demostraciones.

4.10.3 Implementación

Sincronización en el ESP32-CAM

Las funciones de sincronización siguen un patrón común:

1. Verificar conectividad (`isOnline` y `WiFi.status() == WL_CONNECTED`).
2. Cargar los datos locales desde LittleFS.
3. Iterar sobre los registros con `sincronizado = false`.
4. Para cada uno, construir el payload JSON y enviarlo al backend vía HTTP POST.
5. Si el backend responde con éxito (HTTP 200/201), marcar `sincronizado = true` y actualizar el `backend_id` si corresponde.
6. Persistir los cambios en LittleFS.

Para las personas, además del envío de pendientes, se implementa la descarga completa desde el backend (`sincronizarPersonasDesdeBackend()`) que sobrescribe el archivo local si no hay registros locales pendientes, manteniendo el ESP32-CAM actualizado con los datos del servidor.

Sincronización de asistencias por lote

La función `sincronizarAsistencias()` en el ESP32-CAM es especialmente relevante porque maneja el caso más frecuente de datos offline: un trabajador marcó entrada y salida durante el día sin conexión. La función:

1. Carga `asistencias.json` y filtra las que tengan `sincronizado = false`.
2. Construye un JSON con estructura `{"registros": [...]}`.

3. Lo envía a `POST /api/asistencias/sync`.
4. Si el backend retorna `HTTP 200`, marca todos los registros enviados como sincronizados.

Sincronización en el backend

El endpoint `POST /api/asistencias/sync` en `routes/asistencias.py` recibe el lote e itera sobre cada registro, aplicando la verificación de duplicados y realizando la inserción. También dispara el envío a ERP para cada asistencia nueva.

Watchdog de dispositivos

El hilo `device_watchdog` en `mqtt_handler.py` ejecuta dos funciones:

- **Barrido inicial:** Al arrancar el backend, marca todos los dispositivos como **inactivo**. Esto es conservador: un dispositivo debe demostrar que está vivo enviando un heartbeat.
- **Verificación periódica:** Cada 60 segundos, revisa los heartbeats. Los dispositivos que no han enviado heartbeat en más de 90 segundos se marcan como **inactivo**. Esto permite detectar desconexiones incluso si el mensaje LWT no fue entregado (por ejemplo, si el ESP32-CAM se queda sin alimentación de forma abrupta y el broker no alcanza a publicar el LWT).

Herramienta de simulación (`deteccion.py`)

El script `deteccion.py` implementa:

- Conexión directa a la base de datos PostgreSQL.
- Función `extraer_embedding_sim()` con DeepFace (Facenet + RetinaFace + anti-spoofing).
- Función `simular_asistencia_por_foto()` que busca coincidencias del embedding contra la base de datos y, si encuentra una, registra la asistencia en la tabla `asistencias`.
- Menú interactivo en terminal para seleccionar imágenes de la carpeta `static/capturas_prueba/`

4.10.4 Pruebas

Se diseñaron las siguientes pruebas para validar la sincronización diferida, los logs y la integración final:

- **Prueba de sincronización de asistencias offline:** Desconectar el ESP32-CAM de la red, registrar 5 marcaciones de entrada/salida para 2 personas diferentes, reconectar y ejecutar sincronización. Verificar que los 5 registros aparezcan en la base de datos del backend y que el archivo `asistencias.json` local los marque como sincronizados.
- **Prueba de idempotencia de sincronización:** Tras una sincronización exitosa, ejecutar nuevamente la sincronización sin nuevos registros. Verificar que no se inserten registros duplicados en el backend.
- **Prueba de sincronización de personas creadas offline:** Desconectar el dispositivo, registrar una nueva persona (con huella y datos), reconectar y sincronizar. Verificar que la persona aparezca en el backend con un ID definitivo (no `local-`).
- **Prueba de sincronización de turnos y asignaciones:** Desconectar el dispositivo, crear un turno y asignarlo a una persona existente, reconectar y sincronizar. Verificar que el turno y la asignación se repliquen en el backend y que los `backend_id` locales se actualicen correctamente.
- **Prueba de persistencia tras corte de energía:** Durante una sincronización en curso, cortar la alimentación del ESP32-CAM. Al reiniciar, verificar que los archivos JSON no estén corruptos y que los registros con `sincronizado = false` sigan estando disponibles para una nueva sincronización.
- **Prueba de logs de sincronización:** Ejecutar 3 sincronizaciones con diferentes cantidades de registros. Verificar que en la tabla `sincronizacion_log` del backend aparezcan 3 entradas con los conteos correctos de registros enviados y procesados.
- **Prueba de logs biométricos:** Realizar operaciones de registro facial, identificación exitosa, identificación fallida y verificación. Verificar que cada operación genere una entrada en `logs_biometricos` con el tipo de operación y resultado correctos.
- **Prueba de watchdog:** Iniciar el backend, verificar que todos los dispositivos se marquen como inactivos. Conectar un ESP32-CAM, verificar que tras el primer heartbeat cambie a activo. Desconectar el ESP32-CAM y verificar que tras 90 segundos el watchdog lo marque como inactivo.

- **Prueba de simulación facial:** Ejecutar `deteccion.py` y seleccionar una foto de una persona registrada. Verificar que el script encuentre la coincidencia, determine el tipo de evento (entrada/salida) y registre la asistencia en la base de datos. Repetir con una foto de una persona no registrada y verificar que el script indique “Rostro no coincide”.
- **Prueba de flujo completo punta a punta:** Ejecutar el flujo completo: registrar persona con huella y rostro en el ESP32-CAM → sincronizar con backend → verificar que la persona aparezca en el panel web → marcar asistencia por huella offline → reconectar y sincronizar → verificar que la asistencia aparezca en el panel web y se envíe al ERP de prueba.

4.11 Resumen del capítulo

El capítulo 4 presenta el desarrollo completo del prototipo a lo largo de ocho iteraciones, siguiendo la metodología de prototipado evolutivo incremental. Se documenta la integración del hardware (ESP32-CAM, lector AS608, sensor PIR, cámara OV2640), la implementación del almacenamiento local con LittleFS y JSON, el desarrollo del backend con Flask y PostgreSQL (13 tablas con soporte multi-tenant), la comunicación mediante HTTP y MQTT, el reconocimiento facial con DeepFace (modelo Facenet, detector RetinaFace y anti-spoofing), el cifrado de embeddings biométricos con Fernet, la autenticación JWT con roles jerárquicos, el enrolamiento seguro de dispositivos mediante PIN, el sistema antifraude multicapa (PIR + flash PWM + anti-spoofing + cooldown), el panel web para la gestión del dispositivo, la integración con ERP externos vía webhooks con field mapping, y los mecanismos de sincronización diferida con logs de auditoría.

Cada iteración incluyó su respectivo análisis de requisitos, diseño de la solución, implementación detallada y un plan de pruebas de integración para validar el correcto funcionamiento de los componentes desarrollados.

1. Ministerio del Trabajo y Previsión Social, Subsecretaría del Trabajo, Dirección del Trabajo. (2025, abril 25). Resolución 38 Exenta: Establece requisitos obligatorios y procedimiento de autorización para los sistemas electrónicos de registro y control de la asistencia y determinación de las horas de trabajo (modificada por Resolución 9453 Exenta). Biblioteca del Congreso Nacional de Chile. <https://www.bcn.cl/leychile/navegar?idNorma=1203415&idParte=10499950&idVersion=04-25>
2. Defontana Chile. (2024, mayo 9). ¿Qué es la Resolución Exenta N.º 38 en la

- gestión de personas? Defontana Blog. <https://www.defontana.com/blog/cl/que-es-la-resolucion-exenta-38-en-la-gestion-de-personas>
3. Redacción APD. (2025, febrero 3). Qué es un software ERP y para qué sirve: Ventajas y desventajas. APD. <https://www.apd.es/ventajas-y-desventajas-sistema-ERP>
 4. Bernal, A. (2025, marzo 5). Control de asistencia integrado con ERP: Beneficios para empresas. GeoVictoria. <https://www.geovictoria.com/es-cl/blog/control-de-asistencia-integrado-con-ERP-beneficios-para-empresas>
 5. Fernández, A. (2025, abril). Reloj control: Qué es, para qué sirve y qué debes tener en cuenta. Nubox Blog. <https://blog.nubox.com/empresas/reloj-control>
 6. Pachon Robayo, S. (s.f.). Monitoreo de transmisión de video en vivo utilizando ESP32-CAM y Ubidots. Ubidots Centro de Ayuda. Recuperado el 17 de octubre de 2025, de <https://help.ubidots.com/es/articles/7156830-monitoreo-de-transmision-de-video-en-vivo-utilizando-ESP32-CAM-y-ubidots>
 7. Amazon Web Services. (s.f.). ¿Qué es una API? Explicación de qué es una interfaz de programación de aplicaciones. Recuperado el 9 de noviembre de 2025, de <https://aws.amazon.com/es/what-is/api>
 8. Ken, A. (2023, abril 10). Backend: ¿Qué es y para qué sirve? (Falta URL — agregar cuando la tengas)
 9. IONOS. (s.f.). ¿Qué es un frontend? Definición y explicación. Recuperado el 28 de octubre de 2025, de <https://www.ionos.com/es-us/digitalguide/paginas-web/creacion-de-paginas-web/que-es-el-frontend>
 10. Amazon Web Services. (s.f.). ¿Qué es MQTT? Explicación del protocolo MQTT. Recuperado el 6 de octubre de 2025, de <https://aws.amazon.com/es/what-is/mqtt>
 11. Erickson, J. (2024, abril 4). ¿Qué es JSON? Oracle América Latina. Recuperado el 14 de noviembre de 2025, de <https://www.oracle.com/latam/database/what-is-json>
 12. Last Minute Engineers. (s.f.). Empezando con ESP32-CAM: Guía para principiantes. Recuperado el 19 de octubre de 2025, de <https://lastminuteengineers.com/getting-started-with-ESP32-CAM>
 13. UNIT Electronics. (s.f.). Cómo utilizar el lector de huella dactilar AS608. Recuperado el 22 de noviembre de 2025, de <https://blog.uelectronics.com/tarjetas-desarrollo/como-utilizar-el-lector-de-huella-dactilar-AS608>

14. MDN Web Docs. (s.f.). Generalidades del protocolo HTTP. Recuperado el 30 de octubre de 2025, de <https://developer.mozilla.org/es/docs/Web/HTTP/Guides/Overview>
15. Paessler GmbH. (s.f.). ¿Qué es MQTT? – IT Explained. Recuperado el 2 de noviembre de 2025, de <https://www.paessler.com/es/it-explained/mqtt>
16. Docker Inc. (s.f.). ¿Qué es un contenedor? Recuperado el 21 de octubre de 2025, de <https://www.docker.com/resources/what-container>
17. Sunny Valley Cyber Security Inc. (2025, febrero 3). Definición, tipos y ejemplos de servidor de base de datos. Zenarmor. Recuperado el 10 de noviembre de 2025, de <https://www.zenarmor.com/docs/network-basics/what-is-database-server>
18. GeoVictoria. (s.f.). Dispositivos GeoVictoria. Recuperado el 12 de octubre de 2025, de <https://info.geovictoria.com/es-co/dispositivos>
19. ZKTeco Latinoamérica. (s.f.). Tiempo y asistencia. Recuperado el 5 de noviembre de 2025, de <https://zktecolatinoamerica.com/categoria-producto/tiempo-y-asistencia>
20. Talana. (s.f.). Software de asistencia de personal en Chile. Recuperado el 7 de noviembre de 2025, de <https://web.talana.com/software-de-asistencia-de-personal>
21. Buk. (s.f.). Control de asistencia: Gestiona tiempos y alertas. Recuperado el 26 de octubre de 2025, de <https://www.buk.cl/productos/administracion/control-de-asistencia>
22. Defontana. (s.f.). Software y sistema ERP. Recuperado el 18 de noviembre de 2025, de <https://www.defontana.com/cl>
23. Softland Inversiones S.L. (s.f.). Softland Chile: Sistema ERP, contable y de administración. Recuperado el 9 de octubre de 2025, de <https://www.softland.cl>
24. Drumond, C. (s.f.). Scrum: Qué es, cómo funciona y cómo empezar. Atlassian. Recuperado el 3 de noviembre de 2025, de <https://www.atlassian.com/es/agile/scrum>
25. Teach-ICT. (s.f.). Prototipado evolutivo. Recuperado el 22 de octubre de 2025, de https://www.teach-ict.com/as_a2_ict_new/ocr/A2_G063/331_systems_cycle/prototyping_R
26. GeeksforGeeks. (2025, julio 11). Modelo de procesos incrementales – Ingeniería de software. Recuperado el 20 de noviembre de 2025, de <https://www.geeksforgeeks.org/software-engineering/software-engineering-incremental-process-model>
27. Pressman, R. S., & Maxim, B. R. (2021). Ingeniería de software: Un enfoque práctico (9.^a ed.). McGraw-Hill Interamericana.

28. IBM Corporation. (2025). ¿Qué son las pruebas de integración? IBM Think. <https://www.ibm.com/es-es/think/topics/integration-testing>
29. International Organization for Standardization. (2018). ISO 9241-11: Ergonomics of human-system interaction — Part 11: Usability: Definitions and concepts.
30. Nielsen, J. (1993). Usability engineering. Morgan Kaufmann.
31. Brooke, J. (1996). SUS: A “quick and dirty” usability scale. En P. W. Jordan et al. (Eds.), Usability evaluation in industry (pp. 189–194). Taylor & Francis.
32. Bangor, A., Kortum, P. T., & Miller, J. T. (2008). An empirical evaluation of the System Usability Scale. *International Journal of Human-Computer Interaction*, 24(6), 574–594.
33. Microsoft. (2025, julio 10). Introducción a Internet de las cosas (IoT) de Azure. Microsoft Learn. <https://learn.microsoft.com/es-es/azure/iot/iot-introduction>
34. Atlassian. (s.f.). Reliability vs. availability: Key metrics for system performance. Recuperado el 11 de noviembre de 2025, de <https://www.atlassian.com/incident-management/kpis/reliability-vs-availability>
35. Perry, W. E. (2006). Effective methods for software testing (3.^a ed.). Wiley.
36. Kaspersky Lab. (2025). ¿Qué es la biometría? Definición y tipos. <https://latam.kaspersky.com/center/definitions/biometrics>
37. Espressif Systems. (2025). SPIFFS — SPI Flash File System. <https://docs.espressif.com/projects/esp-idf/en/stable/esp32/api-reference/storage/spiffs.html>
38. Amazon Inc. (2025). What is a RESTful API? <https://aws.amazon.com/what-is/restful-api>
39. Red Hat Inc. (2025). ¿Qué son los microservicios? <https://www.redhat.com/es/topics/microservices>
40. EMQX Technologies. (2025). The ultimate guide to MQTT brokers. <https://www.emqx.com/en/blog/ultimate-guide-to-mqtt-broker-comparison>